YAPAY ZEKÂYA GİRİŞ

9. ÜNİTE

BİRİNCİ DERECEDEN MANTIKTA ÇIKARIM

Inference in First-Order Logic

Türkçe Çeviri ve Ayrıntılı Açıklamalı Ders Notları

9.1–9.5 alt bölümleri • 8 özgün kitap şekli • yeni şemalar
çözümlü örnekler • karşılaştırma tabloları • sınav alıştırmaları

İçerik kapsamı: 9.1 Önerme mantığı / FOL çıkarımı; 9.2 Unification ve Lifting; 9.3 Forward Chaining; 9.4 Backward Chaining ve Prolog; 9.5 Resolution, CNF, Skolemization, tamlık ve eşitlik.
Kaynak: Russell ve Norvig, *Artificial Intelligence: A Modern Approach*, 3. baskı, 9. bölüm (kullanıcı tarafından sağlanan metin). Bu dosyada Türkçe aktarım, öğretici açıklamalar ve ayrıca hazırlanan örnekler birlikte sunulur.

İstanbul • Ekim 2026

Eğitim amaçlı çalışma metni. Kitaba ait şekiller, sağlanan kaynak görsellerden alınmıştır.

İçindekiler

9. Birinci Dereceden Mantıkta Çıkarım — Giriş **3**

9.1 Önerme Mantığı ve Birinci Dereceden Mantıkta Çıkarım **3**
9.1.1 Niceleyiciler için çıkarım kuralları . . . 3
Evrensel özelleştirme — Universal Instantiation (UI) . . . 3
Varoluşsal özelleştirme — Existential Instantiation (EI) . . . 4
9.1.2 Önerme mantığındaki çıkarıma indirgeme . . . 4

9.2 Birleştirme ve Yükseltme (Unification and Lifting) **5**
9.2.1 Birinci dereceden bir çıkarım kuralı . . . 6
9.2.2 Birleştirme (Unification) . . . 6
Şekil 9.1 — UNIFY algoritmasının Türkçe sözde kodu . . . 7
9.2.3 Depolama ve geri getirme (Storage and Retrieval) . . . 8

9.3 İleri Zincirleme (Forward Chaining) **8**
9.3.1 Birinci dereceden kesin tümceler (First-Order Definite Clauses) . . . 9
Suç örneği: Albay West suçlu mudur? . . . 9
9.3.2 Basit bir ileri zincirleme algoritması . . . 9
Şekil 9.3 — FOL-FC-ASK sözde kodu . . . 9
Crime örneği: ilk iterasyon . . . 10
İkinci iterasyon . . . 10
Sağlamlık, tamlık ve sonlanma . . . 11
9.3.3 Verimli ileri zincirleme . . . 11
Kural-gerçek eşleştirme ve koşul sırası (Conjunct Ordering) . . . 11
Artımlı ileri zincirleme (Incremental Forward Chaining) . . . 13
Rete algoritması ve üretim sistemleri . . . 13
İlgisiz çıkarımlar ve Magic Sets . . . 13

9.4 Geri Zincirleme (Backward Chaining) **14**
9.4.1 Bir geri zincirleme algoritması . . . 14
Şekil 9.6 — Geri zincirleme algoritması . . . 14
Şekil 9.7 — West suç örneğinin geri zincirleme ispat ağacı . . . 15
9.4.2 Mantıksal programlama (Logic Programming) . . . 15
Liste birleştirme — Append ilişkisi . . . 16
Prolog'un uygulamadaki tercihleri . . . 16
9.4.3 Mantıksal programların verimli uygulanması . . . 16
Şekil 9.8 — Derlenmiş Append yordamı . . . 17
9.4.4 Gereksiz çıkarım ve sonsuz döngüler . . . 17
9.4.5 Prolog'un veritabanı semantiği . . . 19
9.4.6 Kısıt mantığı programlaması (Constraint Logic Programming) . . . 19

9.5 Çözümleme (Resolution) **20**
9.5.1 Birinci dereceden mantık için CNF . . . 20
«Bütün hayvanları seven herkes birisi tarafından sevilir» örneği . . . 21
9.5.2 Resolution çıkarım kuralı . . . 22
9.5.3 Örnek çözümleme ispatları . . . 23
Birinci ispat: Albay West suçlu mudur? (Şekil 9.11) . . . 23
İkinci ispat: Curiosity Tuna adlı kediyi öldürdü mü? (Şekil 9.12) . . . 24

Soru «Kediyi kim öldürdü?» olursa . . . 26
9.5.4 Resolution'ın tamlığı (Completeness of Resolution) . . . 26
Herbrand evreni, doyurma ve Herbrand tabanı . . . 27
Lifting Lemma . . . 28
Yan kutu — Gödel'in Eksiklik Teoremi . . . 28
9.5.5 Eşitlik (Equality) . . . 29
Eşitlik aksiyomları . . . 29
Demodulation (yönlü yeniden yazma) . . . 29
Paramodulation . . . 29
Equational Unification . . . 30
9.5.6 Çözümleme stratejileri (Resolution Strategies) . . . 30
Unit Preference / Unit Resolution . . . 30
Set of Support (destek kümesi) . . . 30
Input Resolution ve Linear Resolution . . . 30
Subsumption (kapsama yoluyla eleme) . . . 31
Resolution teorem ispatlayıcılarının uygulamaları . . . 31

Ek A — Bütün konuları birleştiren çözümlü örnekler 31
A.1 Evrensel ve varoluşsal niceleyici . . . 31
A.2 Genelleştirilmiş Modus Ponens ve bağlama aktarımı . . . 31
A.3 Birleştirme testi: altı farklı durum . . . 32
A.4 İleri zincirleme: dört kural, üç tur . . . 32
A.5 Geri zincirleme: aynı problemde hedeften gerçeğe . . . 33
A.6 CNF dönüşümü — kısa fakat tam örnek . . . 33
A.7 Resolution ile çelişki ispatı . . . 33
A.8 Horn tümcesi ile genel tümce farkı . . . 33
A.9 Subsumption ve indeksleme . . . 33
A.10 Eşitlik ve yeniden yazma . . . 34
A.11 Tabled logic programming neden yararlı? . . . 34
A.12 Sınav tipi bütünleşik soru . . . 34

Ek B — Hızlı tekrar, kavram sözlüğü ve kontrol soruları 34
B.1 Karar rehberi: hangi yöntem ne zaman? . . . 34
B.2 Kavram sözlüğü . . . 35
B.3 Kendi kendini değerlendirme soruları . . . 36
B.4 Bölümün ana sonucu . . . 37

9. Birinci Dereceden Mantıkta Çıkarım — Giriş

Bu ünitede birinci dereceden mantıkla (First-Order Logic, **FOL**) ifade edilmiş soruları cevaplamak üzere etkili çıkarım prosedürlerini ele alıyoruz. Daha önce önerme mantığında sağlam (*sound*) ve tam (*complete*) çıkarımların nasıl yapılabildiğini görmüştük. Şimdi bu sonuçları niceleyicili, değişkenli, nesneler arası ilişkiler kurabilen bir mantığa genişleteceğiz.

Önce evrensel ve varoluşsal niceleyicilere ilişkin çıkarım kurallarını inceleyecek, ardından birinci dereceden çıkarımı önerme mantığına indirgeme fikrini göreceğiz. Bu yöntem sonsuz sayıda örnek üretme riski taşıyabildiğinden, ifadeleri doğrudan eşleştirebilen **unification (birleştirme)** ve **lifting (yükseltme)** kavramlarına geçeceğiz. Daha sonra üç çıkarım ailesini ele alacağız: **Forward Chaining**, **Backward Chaining** ve genel ifadeler için **Resolution**.

İleri ve geri zincirleme çok verimli olabilir; ancak burada işlenen biçimleri, esas olarak *Horn tümceleri* ile ifade edilebilen bilgi tabanlarına uygulanır. Genel birinci dereceden mantık ifadelerinde Resolution tabanlı teorem ispatlama gerekir.

Temel hatırlatma. Bilgi tabanı KB için $KB \models \alpha$ yazımı, bilgi tabanını doğru yapan her yorumda α ifadesinin de doğru olduğunu belirtir. Çıkarım mekanizmasının doğru sonuçlar bulmasına sağlamlık, çıkarsanabilen her hedefi ispatlayabilmesine tamlık denir. Ancak tamlık, algoritmanın bütün olumsuz sorgularda sonlanacağını garanti etmez.

9.1 Önerme Mantığı ve Birinci Dereceden Mantıkta Çıkarım

İlk adım, niceleyici içeren ifadelerden niceleyici içermeyen ifadeler üretmektir. Böylece daha önce öğrendiğimiz önerme mantığı algoritmalarından yararlanabiliriz. Ardından bu yöntemin neden pahalı olduğunu ve daha kısa yolun nasıl bulunacağını göstereceğiz.

9.1.1 Niceleyiciler için çıkarım kuralları

Evrensel özelleştirme — Universal Instantiation (UI)

Bilgi tabanımızın «bütün açgözlü krallar kötüdür» aksiyomunu içerdiğini varsayalım:

$$\forall x\ [King(x) \land Greedy(x) \Rightarrow Evil(x)].$$

Bu genel ifade John, Richard veya John'un babası için somutlaştırılabilir:

$$\begin{aligned} King(John) \land Greedy(John) &\Rightarrow Evil(John), \\ King(Richard) \land Greedy(Richard) &\Rightarrow Evil(Richard), \\ King(Father(John)) \land Greedy(Father(John)) &\Rightarrow Evil(Father(John)). \end{aligned}$$

UI kuralına göre evrensel niceleyicili değişkenin yerine **ground term (değişken içermeyen terim)** koyarak bir örnek üretilebilir. $SUBST(\theta, \alpha)$, θ yerine koymasının α üzerine uygulanmış biçimini belirtir:

$$\boxed{\frac{\forall v\ \alpha}{SUBST(\{v/g\}, \alpha)}} \qquad (g \text{ ground terim}).$$

Yukarıdaki üç özel örnek sırasıyla $\{x/John\}$, $\{x/Richard\}$ ve $\{x/Father(John)\}$ ile elde edilir. Bir kuralın John'a uygulanması, John'un gerçekten açgözlü veya kral olduğu sonucunu

tek başına vermez; yalnızca koşullu ifadeyi John'a özelleştirir.

Çözümlü örnek. $\forall x\,[Bird(x) \Rightarrow Animal(x)]$ ve $Bird(Tweety)$ biliniyor olsun. Önce UI ile $Bird(Tweety) \Rightarrow Animal(Tweety)$ elde edilir; ardından Modus Ponens kullanılarak $Animal(Tweety)$ kanıtlanır. İki adımı karıştırmamak gerekir.

Varoluşsal özelleştirme — Existential Instantiation (EI)

Varoluşsal niceleyicili bir ifadede değişkenin yerine, bilgi tabanında daha önce kullanılmamış **yeni bir sabit** konur:

$$\boxed{\frac{\exists v\ \alpha}{SUBST(\{v/k\}, \alpha)}} \qquad (k \text{ yeni sabit}).$$

Örneğin «John'un başında bir taç bulunmaktadır» ifadesi:

$$\exists x\,[Crown(x) \wedge OnHead(x, John)]$$

şu ifadeyle somutlaştırılabilir:

$$Crown(C_1) \wedge OnHead(C_1, John).$$

Buradaki C_1 daha önce başka bir nesneyi ifade etmemelidir. Yeni isme **Skolem sabiti** denir. Matematikte belirli bir niteliği olan yeni bir sayıya e adı verip, zaten farklı bir nesneyi gösteren π adını vermemek buna benzer. EI, sonraki bölümdeki Skolemleştirme işleminin özel durumudur.

UI birçok farklı sabit için tekrar uygulanabilir. EI ise belirli bir varoluşsal ifade için taze tanık sabiti oluşturularak bir kez uygulanır ve varoluşsal ifade yeni bilgi tabanında kullanılmadan çıkarılabilir. Örneğin $\exists x\ Kill(x, Victim)$ ifadesi $Kill(Murderer, Victim)$ biçiminde adlandırılabilir. Yeni bilgi tabanı eski bilgi tabanıyla tam mantıksal eşdeğer değildir; fakat **tatmin edilebilirlik** korunur (bu bağlamda çıkarımsal eşdeğerlik).

Evrensel özelleştirme	Varoluşsal özelleştirme
$\forall x$: bütün nesneler için	$\exists x$: en az bir nesne için
Bilinen herhangi bir ground terim kullanılır	Daha önce kullanılmamış sabit kullanılır
Birçok kez uygulanabilir	Bir tanık oluşturmak üzere bir kez uygulanır
«Herkese ait kuralı Ali'ye uygula»	«Var olan ama kimliği bilinmeyen birine ad ver»

Önemli ayrım. Burada kullanılan yerine koyma sözdizimsel bir işlemdir: değişken, bir terimle değiştirilir. Mantığın semantiğindeki yorum ise değişkenlere evrendeki nesneleri atar; bu ikisi karıştırılmamalıdır.

9.1.2 Önerme mantığındaki çıkarıma indirgeme

Niceleyicileri özelleştirebildiğimize göre birinci dereceden bilgiyi önerme mantığına indirgeyebiliriz. Varoluşsal ifade bir kez özelleştirilirken evrensel ifade, olası bütün ground örnekleriyle temsil edilebilir.

Bilgi tabanımız:

$$\begin{gathered}\forall x\,[King(x) \wedge Greedy(x) \Rightarrow Evil(x)],\\ King(John), \qquad Greedy(John), \qquad Brother(Richard, John).\end{gathered} \tag{9.1}$$

Olası sabitler John ve Richard ise genel kuralı iki kez özelleştiririz:

$$\begin{gathered}King(John) \wedge Greedy(John) \Rightarrow Evil(John),\\ King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard).\end{gathered}$$

Böylece $King(John)$ ve $Greedy(John)$ gibi ground atomları tek tek önerme sembolleriymiş gibi düşünebiliriz. Önerme mantığının tam algoritmaları kullanıldığında $Evil(John)$ sonucu çıkar; ancak Richard için kral veya açgözlü olma bilgisi verilmediğinden $Evil(Richard)$ ispatlanmaz.

Bu yaklaşıma **propositionalization (önermeselleştirme)** denir. Genelde birinci dereceden bilgi tabanı ve sorgu, mantıksal gerektirmeyi koruyan bir önerme mantığı problemine dönüştürülebilir. Ne var ki bilgi tabanında fonksiyon sembolü varsa bütün ground örneklerin kümesi sonsuz olabilir:

$$John,\ Father(John),\ Father(Father(John)),\ \ldots$$

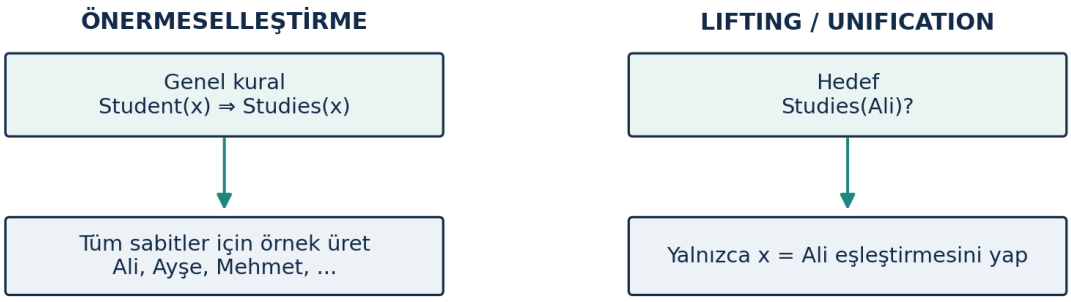
Herbrand teoreminin temel fikri şöyledir: Bir ifade bilgi tabanından çıkıyorsa ispatı için önermeselleştirilmiş bilginin **sonlu** bir alt kümesi yeterlidir. Bu yüzden önce sabitler, sonra fonksiyon iç içe geçme derinliği 1 olan terimler, sonra derinliği 2 olan terimler vb. denenebilir. İspat gerçekten varsa sonlu bir derinlikte bulunur; bulunmuyorsa arama sonsuza dek devam edebilir.

Fonksiyon terimi	Derinlik
$John$	0
$Father(John)$	1
$Father(Father(John))$	2
$Father(Father(Father(John)))$	3

Bu durum birinci dereceden mantıkta **semidecidability (yarı karar verilebilirlik)** sonucuna götürür: mantıksal olarak gerektirilen her sonuç için sonunda «evet» diyen yöntemler vardır, fakat gerektirilmeyen bütün sonuçlara da her durumda sonlu sürede «hayır» diyebilen genel bir karar yöntemi yoktur. Turing ve Church 1936'da bu sınırlamanın kaçınılmazlığını farklı yollardan göstermişlerdir. Burada «henüz ispat bulamadım» ile «yanlıştır» aynı değildir.

9.2 Birleştirme ve Yükseltme (Unification and Lifting)

Önermeselleştirme gereksiz örnekler üretir. $Evil(x)$ sorgusunu cevaplamak isterken Richard için hiçbir işe yaramayan koşullu ifadeler üretmemize gerek yoktur. Amaç doğrudan ilgili değişkenleri eşleştirmektir.



9.2.1 Birinci dereceden bir çıkarım kuralı

$King(John)$ ve $Greedy(John)$ bilgileri varsa genel açgözlü kral kuralına $\theta = \{x/John\}$ uygulanır ve $Evil(John)$ üretilir. Hatta $Greedy(John)$ yerine $\forall y \;\; Greedy(y)$ bilinse de $\theta = \{x/John, y/John\}$ eşleştirmesiyle aynı sonuca ulaşılır.

Bu düşünce **Generalized Modus Ponens (Genelleştirilmiş Modus Ponens)** kuralını verir. p_i', p_i ve q atomik ifadeler, $SUBST(\theta, p_i') = SUBST(\theta, p_i)$ bütün i değerleri için geçerliyse:

$$\boxed{\frac{p_1', \ldots, p_n', \quad (p_1 \wedge \cdots \wedge p_n \Rightarrow q)}{SUBST(\theta, q)}}.$$

Suç / kral örneğinde $p_1' = King(John)$, $p_2' = Greedy(y)$, $p_1 = King(x)$, $p_2 = Greedy(x)$, $q = Evil(x)$ olur. $\theta = \{x/John, y/John\}$ bütün öncülleri aynı hâle getirir ve sonuç $Evil(John)$ olur.

Sağlamlık gerekçesi. Evrensel özelleştirme nedeniyle $p \models SUBST(\theta, p)$; dolayısıyla uygun yerine koymalarla öncüller aynılaşır. Eşleşen öncüllere sıradan Modus Ponens uygulandığında sonuç elde edilir. Generalized Modus Ponens bu nedenle sağlamdır.

Bu işlem **lifting** olarak adlandırılır: değişkensiz Modus Ponens, birinci dereceden değişkenli mantığa yükseltilmiştir. Aynı fikir ileri zincirleme, geri zincirleme ve çözümleme yöntemlerine de uygulanabilir. En büyük kazanç sadece **gereken** özelleştirmelerin yapılmasıdır.

9.2.2 Birleştirme (Unification)

İki mantıksal ifadeyi aynı hâle getiren bir yerine koyma aranmasına **unification** denir. Algoritma uygun bir θ bulursa:

$$\boxed{UNIFY(p, q) = \theta \quad \text{öyle ki} \quad SUBST(\theta, p) = SUBST(\theta, q).}$$

$Knows(John, x)$ sorgusunun («John kimi tanır?») farklı bilgi tabanı ifadeleriyle eşleştirilmesi:

Birleştirilecek ifade	Sonuç
$Knows(John, Jane)$	$\{x/Jane\}$
$Knows(y, Bill)$	$\{x/Bill, y/John\}$
$Knows(y, Mother(y))$	$\{y/John, x/Mother(John)\}$

Birleştirilecek ifade	Sonuç
$Knows(x, Elizabeth)$	Aynı değişken adı yüzünden ilk denemede başarısız

Son satırda iki ifadede de x kullanılması, x'in hem John hem Elizabeth olmasını istiyormuş gibi yanlış çakışma yaratır. Oysa $Knows(x, Elizabeth)$ ifadesinin evrensel yorumunda herkes Elizabeth'i tanımaktadır. Çözüm, bu bağımsız x'i x_{17} olarak yeniden adlandırmaktır (**standardizing apart**):

$$UNIFY(Knows(John, x), Knows(x_{17}, Elizabeth)) = \{x/Elizabeth, x_{17}/John\}.$$

Most General Unifier (MGU). Birleştirici tek olmayabilir. $Knows(John, x)$ ile $Knows(y, z)$ için $\{y/John, x/z\}$ ve $\{y/John, x/John, z/John\}$ ikisi de kullanılabilir. İlki daha az kısıtlayıcı olduğundan **en genel birleştirici**dir. İkinci çözüm, ilkine $\{z/John\}$ eklenerek elde edilir. Birleştirilebilir iki ifade için MGU, değişkenlerin yeniden adlandırılması bakımından tektir.

Şekil 9.1 — UNIFY algoritmasının Türkçe sözde kodu

```
UNIFY(x, y, θ):
    θ başarısızsa: failure
    x = y ise: θ
    x değişkense: UNIFY-VAR(x, y, θ)
    y değişkense: UNIFY-VAR(y, x, θ)
    x ve y bileşik ifadelerse:
        önce OP (işlem sembolü), sonra ARGS (argümanlar)
        üzerinde özyinelemeli UNIFY uygula
    x ve y listelerse:
        önce FIRST, sonra REST parçalarını birleştir
    diğer durumlarda: failure

UNIFY-VAR(var, x, θ):
    var önceden bağlıysa: mevcut değeri x ile birleştir
    x önceden bağlıysa: var'ı mevcut değerle birleştir
    var, x teriminin içinde geçiyorsa: failure
    aksi hâlde: θ'ya {var/x} ekle
```

Algoritma iki ifadenin yapısını eşzamanlı inceler, her eşleşmeden sonra θ'yı genişletir. $F(A, B)$ bileşik ifadesinde OP = F, ARGS = (A,B) olur. Daha önce bağlanmış değişkenlerin sonraki karşılaştırmalarda tutarlı kalması gerekir.

Occur check. $P(x)$ ile $P(f(x))$ birleştirilmeye çalışılırsa $x = f(x)$ koşulu gerekir. Fakat x, atanmak istenen terimin içinde bulunmaktadır; standart **sonlu terim** mantığında bu mümkün değildir. $S(x)$ ile $S(S(x))$ de aynı nedenle birleştirilemez. Klasik uygulamada kontrol maliyetli olabilir. Bazı Prolog uygulamaları performans için occur check'i varsayılan olarak atlar; bu durumda standart sonlu-terim yorumuna göre sağlam olmayan sonuçlar ortaya çıkabilir.

Çözümlü örnek.

$$UNIFY(P(x, f(y), Ali), P(Mehmet, f(Ayse), z))$$

Önce x = Mehmet; sonra f(y) ile f(Ayse) eşleştiğinden y = Ayse; son olarak z = Ali olur. Böylece

$\theta = \{x/Mehmet, y/Ayse, z/Ali\}$ ve iki ifade de $P(Mehmet, f(Ayse), Ali)$ hâline gelir.

9.2.3 Depolama ve geri getirme (Storage and Retrieval)

Bilgi tabanına bilgi eklemekte kullanılan TELL ile soru sormada kullanılan ASK'in altında iki temel işlem vardır: **STORE(s)**, s ifadesini depolar; **FETCH(q)** ise q ile bilgi tabanındaki bazı ifadeleri eşleştiren bütün birleştiricileri getirir. Örneğin $FETCH(Knows(John, x))$, John'un tanıdığı kişilere ilişkin eşleştirmeleri bulur.

En basit uygulama bütün gerçekleri tek listede saklayıp sorguyu her gerçekle birleştirmektir. Bu doğru çalışır fakat verimsizdir. **Predicate indexing (yüklem indeksleme)** ile bütün `Knows` kayıtları bir kovada, bütün `Brother` kayıtları başka kovada toplanır ve kovalar hash tablosuyla erişilebilir hâle getirilir. Böylece $Knows(John, x)$ sorgusu için `Brother(Richard,John)` denemesi yapılmaz.

Yalnız yükleme göre indeksleme bazen yetersizdir. $Employs(x, y)$ milyonlarca kişi için bulunuyorsa $Employs(x, Richard)$ sorgusunda ikinci argümana göre, $Employs(IBM, y)$ sorgusunda ilk argümana göre erişmek faydalıdır. Aynı gerçek birden fazla indeks anahtarı altında bulunabilir.

Örneğin $Employs(IBM, Richard)$ gerçeği şu sorguları karşılar:

- $Employs(IBM, Richard)$: IBM Richard'ı çalıştırıyor mu?
- $Employs(x, Richard)$: Richard'ı kim çalıştırıyor?
- $Employs(IBM, y)$: IBM kimi çalıştırıyor?
- $Employs(x, y)$: Kim kimi çalıştırıyor?

Bu genel-özel sorgu düzenine **subsumption lattice (kapsama kafesi)** denir. Aşağıdaki kavramsal gösterimde en genel sorgu en üstte, en özel sorgu en alttadır:

$$
\begin{array}{ccc}
 & Employs(x, y) & \\
 \swarrow & & \searrow \\
Employs(x, Richard) & & Employs(IBM, y) \\
 \searrow & & \swarrow \\
 & Employs(IBM, Richard) &
\end{array}
$$

Kitaptaki Şekil 9.2(b), $Employs(John, John)$ gibi aynı sabitin tekrarlandığı durumlarda $Employs(x, x)$ düğümünün de ortaya çıktığını gösterir. Genel kafeste bir alt düğüm, üst düğümün özelleştirilmesiyle elde edilir; iki düğümün en genel ortak özelleştirmesi unification ile ilgilidir.

n argümanlı yüklem için kafesteki basit değişken/sabit düzenleri $O(2^n)$ ölçüsünde büyüyebilir; fonksiyon terimleri varsa yapı terim büyüklüğüne göre de üstel artabilir. Her olası indeks tutulursa depolama ve güncelleme maliyeti aşırı yükselir. Pratik sistemler sabit kurallarla belirli sütunları indeksler ya da sorgu örüntülerine göre **uyarlamalı indeksleme** uygular. Büyük ticari veritabanlarında bu denge özel önem taşır.

9.3 İleri Zincirleme (Forward Chaining)

7.5. bölümde önerme mantığındaki kesin tümceler için verilen ileri zincirleme, bilgi tabanında zaten bulunan atomik gerçeklerden başlar. Modus Ponens ileri yönde uygulanır, yeni gerçekler eklenir ve yeni bir sonuç üretilemeyene kadar işlem sürer. Özellikle «durum $\Rightarrow$ tepki» tipindeki

kurallar, yeni bilgi geldiğinde harekete geçen sistemler için kullanışlıdır.

9.3.1 Birinci dereceden kesin tümceler (First-Order Definite Clauses)

Definite clause (kesin tümce) tam bir pozitif literal içeren tümcedir; ya doğrudan bir atomik ifade ya da pozitif atomların birleşiminden tek pozitif sonuç çıkaran bir kuraldır:

$$King(x) \land Greedy(x) \Rightarrow Evil(x), \quad King(John), \quad Greedy(y).$$

Birinci dereceden mantık değişkenlerine izin verir; bu değişkenler evrensel niceleyicili kabul edilir ve pratikte $\forall$ çoğu kez yazılmaz. Tek pozitif literal kısıtı nedeniyle her genel FOL bilgi tabanı kesin tümcelerle temsil edilemez; ancak pek çok kural tabanlı sistem bu biçime uygundur.

Suç örneği: Albay West suçlu mudur?

Amerikan yasasına göre bir Amerikalının düşman ülkeye silah satması suçtur. Amerika'nın düşmanı Nono'nun bazı füzeleri vardır. Bu füzelerin tümünü Amerikalı Albay West Nono'ya satmıştır. West'in suçlu olduğunu kanıtlayacağız.

Cümleleri birinci dereceden kesin tümcelere dönüştürelim:

$$\begin{aligned}
&American(x) \land Weapon(y) \land \\
&Sells(x, y, z) \land Hostile(z) \Rightarrow Criminal(x) && (9.3)\\
Owns(Nono, M_1) \quad & (9.4)\\
Missile(M_1) \quad & (9.5)\\
Missile(x) \land Owns(Nono, x) &\Rightarrow Sells(West, x, Nono) && (9.6)\\
Missile(x) &\Rightarrow Weapon(x) && (9.7)\\
Enemy(x, America) &\Rightarrow Hostile(x) && (9.8)\\
American(West) \quad & (9.9)\\
Enemy(Nono, America). \quad & (9.10)
\end{aligned}$$

Burada $Sells(x, y, z)$, «x, y nesnesini z'ye satar» anlamındadır. $\exists x\,[Owns(Nono, x) \land Missile(x)]$ varoluşsal ifadesine EI uygulanarak taze M_1 sabiti seçildi ve iki başlangıç gerçeği üretildi. «Füzeler silahtır» ve «Amerika'nın düşmanı hasımdır» şeklindeki arka plan kuralları da sonuca varabilmek için gereklidir.

Bu bilgi tabanında fonksiyon sembolleri bulunmaz. **Datalog**, fonksiyon sembolü içermeyen birinci dereceden kesin tümcelerle sınırlı dildir. İlişkisel veritabanı benzeri bilgileri rahat ifade eder ve olası ground atomların sayısı sonlu olduğundan çıkarım kolaylaşır.

9.3.2 Basit bir ileri zincirleme algoritması

FOL-FC-ASK mevcut gerçeklerle öncülleri eşleşen kuralları bulur, bütün yeni sonuçları oluşturur ve sorgu cevaplanıncaya veya sabit noktaya ulaşılıncaya kadar devam eder. Yalnız değişken adı değiştirilerek oluşturulan ifadeler yeni gerçek sayılmaz: $Likes(x, IceCream)$ ile $Likes(y, IceCream)$ aynı evrensel iddiadır.

Şekil 9.3 — FOL-FC-ASK sözde kodu

```
FOL-FC-ASK(KB, α)  # KB: kesin tümceler; α: atomik sorgu
```

```
repeat:
    new = {}
    KB'deki her implication kuralı (p1 ∧ ... ∧ pn ⇒ q) için:
        Kural değişkenlerini yeni isimlerle ayır.
        Öncülleri KB gerçekleriyle eşleştiren bütün θ'leri bul.
        q' = SUBST(θ, q)
        q' KB'de veya new içinde bilinenle birleşmiyorsa:
            new'e q' ekle.
            φ = UNIFY(q', α)
            φ başarılıysa cevabı (φ) döndür.
    new içindeki bütün gerçekleri KB'ye ekle.
until new boş
false döndür
```

Bu algoritma öğretmek için basittir, ama her döngüde pek çok kuralı tekrar kontrol eder. Yeni gerçeğin sorguyla birleştirilmesi, yalnızca bir cevap isteniyorsa erken durmaya imkân verir.

Crime örneği: ilk iterasyon

(9.3) numaralı suç kuralının bazı öncülleri henüz sağlanmaz. Öte yandan:

1. $Missile(M_1)$ ve $Owns(Nono, M_1)$ kullanılarak (9.6)'dan $Sells(West, M_1, Nono)$ çıkarılır.
2. $Missile(M_1)$ kullanılarak (9.7)'den $Weapon(M_1)$ çıkarılır.
3. $Enemy(Nono, America)$ kullanılarak (9.8)'den $Hostile(Nono)$ çıkarılır.

Dolayısıyla ilk turun yeni gerçekleri:

$$\boxed{\{Sells(West, M_1, Nono),\ Weapon(M_1),\ Hostile(Nono)\}}.$$

İkinci iterasyon

Artık suç kuralının dört öncülü de eşleşir:

Gereken atom	Bilinen gerçek
$American(x)$	$American(West)$
$Weapon(y)$	$Weapon(M_1)$
$Sells(x, y, z)$	$Sells(West, M_1, Nono)$
$Hostile(z)$	$Hostile(Nono)$

$\theta = \{x/West, y/M_1, z/Nono\}$ ile $Criminal(West)$ çıkar. Başka yeni bilgi kalmadığında **fixed point (sabit nokta)** elde edilir: ileri zincirlemenin üretebileceği bütün atomik gerçekler zaten bilgi tabanındadır.

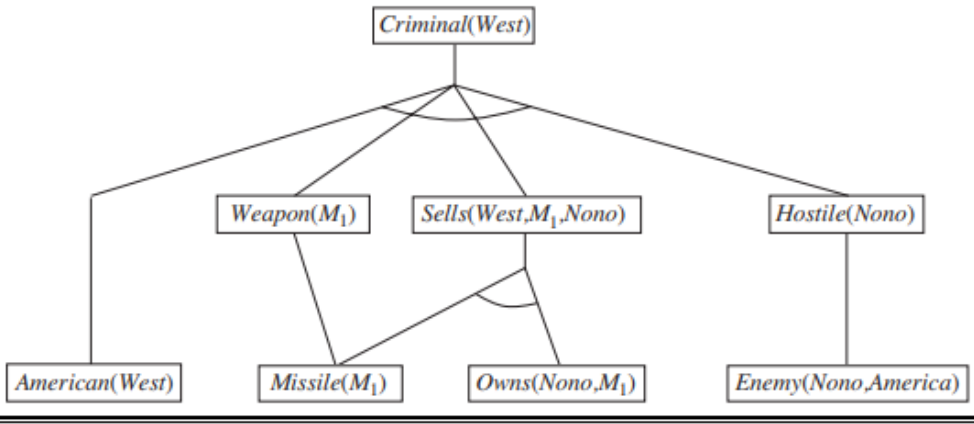


Figure 9.4 The proof tree generated by forward chaining on the crime example. The initial facts appear at the bottom level, facts inferred on the first iteration in the middle level, and facts inferred on the second iteration at the top level.

Şekil 9.4'ü okuma. En altta $American(West)$, $Missile(M_1)$, $Owns(Nono, M_1)$ ve $Enemy(Nono, America)$ vardır. Orta katta satış, silah olma ve hasımlık sonuçları çıkarılır. En üstte dört öncül birleşerek $Criminal(West)$ verir. Yukarıdan bakınca tek bir hedef görülse de ağacın kurulma yönü **aşağıdan yukarıyadır**.

Sağlamlık, tamlık ve sonlanma

İleri zincirleme her adımda sağlam olan Generalized Modus Ponens'i kullandığından **sağlamdır**. Definite clause bilgi tabanları için de **tamdır**: mantıksal olarak çıkan bir atomik sonuç aramayla bulunabilir.

Datalog'da en fazla k argümanlı p yüklem ve n sabit varsa ground atom sayısı en çok $p\,n^k$ düzeyindedir ($n \geq 1$). Örneğin $p = 4, n = 3, k = 2$ için 36 atomluk üst sınır bulunur; dolayısıyla yeni ekleme yapan iterasyon sayısı sonludur. Ancak fonksiyon sembolleri varsa:

$$NatNum(0), \quad NatNum(n) \Rightarrow NatNum(S(n))$$

kuralları $NatNum(S(0)), NatNum(S(S(0))), \ldots$ ifadelerini sonsuza kadar üretir. Bu durumda hedef çıkarılıyorsa ispat sonlu adımda bulunabilir; cevap yoksa sonlanma garanti değildir. Genel definite clause gerektirmesi de yarı karar verilebilirdir.

9.3.3 Verimli ileri zincirleme

Şekil 9.3'teki basit algoritmanın üç verimsizlik kaynağı vardır: (i) kural öncüllerini gerçeklerle eşleştirme (**pattern matching**), (ii) değişmeyen kuralları her tur yeniden kontrol etme, (iii) hedefle ilgisiz sonuçları üretme. Şimdi bunları ayrı ayrı inceleyelim.

Kural-gerçek eşleştirme ve koşul sırası (Conjunct Ordering)

$Missile(x) \Rightarrow Weapon(x)$ kuralında sadece Missile kayıtlarının bulunması yeterlidir. Ancak:

$$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(West, x, Nono)$$

kuralında ortak x için iki koşul sağlanmalıdır. Nono'nun sahip olduğu 10.000 nesne, fakat tüm veritabanında sadece 20 füze varsa önce füzeleri bulup sahipliği kontrol etmek genellikle daha

az maliyetlidir. Önce hangi birleşim bileşeninin deneneceğini seçmeye **conjunct ordering** denir. En iyi sıralama NP-hard olabilir; iyi sezgiseller, örneğin 6. bölümdeki **MRV** (en az kalan değer) fikri, daha küçük aday kümesini önce ele alır.

Örüntü eşleştirme ile **Constraint Satisfaction Problem (CSP)** arasında yakın bağ vardır: $Missile(x)$ x üzerinde tek değişkenli, $Diff(x, y)$ ise iki değişkenli kısıttır. Sonlu alanlı bir CSP uygun ground bilgilerle tek bir kesin tümce olarak temsil edilebilir.

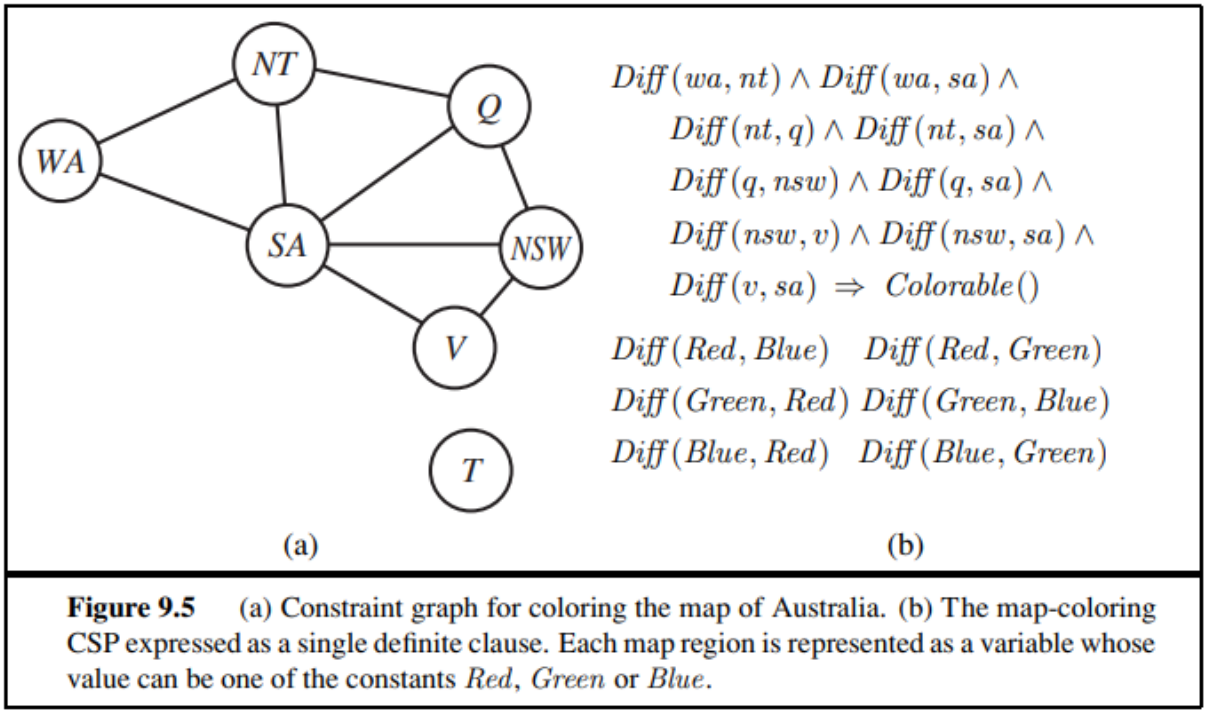


Figure 9.5 (a) Constraint graph for coloring the map of Australia. (b) The map-coloring CSP expressed as a single definite clause. Each map region is represented as a variable whose value can be one of the constants *Red*, *Green* or *Blue*.

Şekil 9.5'in okunması. WA = Batı Avustralya, NT = Kuzey Toprakları, SA = Güney Avustralya, Q = Queensland, NSW = Yeni Güney Galler, V = Victoria, T = Tazmanya. İki bölge arasında çizgi varsa renklerinin farklı olması gerekir. Her bölgenin alanı $\{Red, Green, Blue\}$; farklı renkli sıralı çiftler için altı `Diff` gerçeği vardır.

Tek kesin tümce şu koşulları birlikte ister:

$$Diff(wa, nt) \wedge Diff(wa, sa) \wedge Diff(nt, q) \wedge Diff(nt, sa) \\ \wedge Diff(q, nsw) \wedge Diff(q, sa) \wedge Diff(nsw, v) \\ \wedge Diff(nsw, sa) \wedge Diff(v, sa) \Rightarrow Colorable().$$

Ground gerçeklerden bazıları $Diff(Red, Blue)$, $Diff(Red, Green)$, $Diff(Green, Red)$, $Diff(Green, Blue)$, $Diff(Blue, Red)$ ve $Diff(Blue, Green)$ şeklindedir. CSP çözülebiliyorsa $Colorable()$ çıkarılabilir. Örnek geçerli boyama: WA=Red, NT=Green, SA=Blue, Q=Red, NSW=Green, V=Red; T komşusuz olduğundan üç renkten biri olabilir. Komşu çiftlerin her biri farklı renk taşır.

Genel CSP, 3-SAT'i özel durum olarak içerdiğinden, kural eşleştirmenin en genel biçimi NP-hard'dır. Yine de gerçek sistemlerde kurallar genellikle küçük ve sabit genişliktedir: kural uzunluğu ve yüklem aritesi sabit kabul edilirse yalnız ground gerçek sayısına bağlı **veri karmaşıklığı** polinomiyeldir. Kısıt grafiği ağaç olduğunda da eşleştirme çok daha kolaydır. SA düğümü çıkarılırsa kenarlar $wa - nt - q - nsw - v$ zincirini oluşturur:

$$Diff(wa, nt) \wedge Diff(nt, q) \wedge Diff(q, nsw) \wedge Diff(nsw, v) \Rightarrow Colorable().$$

Ağaç yapılı CSP çözüm algoritmaları bu tür kural eşleştirmelerine doğrudan uygulanabilir.

Artımlı ileri zincirleme (Incremental Forward Chaining)

İkinci turda $Missile(x) \Rightarrow Weapon(x)$ kuralı tekrar $Missile(M_1)$ ile eşleşir; ama sonuç $Weapon(M_1)$ zaten bilinir. Gereksiz kontrolü önlemek için şu gözlemi kullanırız:

> t. turda yeni türetilen bir gerçek, en azından $(t-1)$. turda yeni eklenmiş bir gerçeğe dayanmak zorundadır; aksi hâlde önceki turda da türetilebilirdi.

Artımlı yaklaşım, sadece önceki turda yeni gelen bir gerçeğin öncüllerinden biriyle eşleştiği kuralları tekrar kontrol eder. Diğer koşullar eski gerçeklerden karşılanabilir. Böylece her turda üretilen yeni gerçekler değişmeden gereksiz iş azaltılır. Bazı sistemler her TELL çağrısıyla ilgili kuralları tetikler ve sabit noktaya kadar zinciri sürdürür.

Rete algoritması ve üretim sistemleri

Bir kuralın bazı öncülleri ilk turda eşleşip kalanları daha sonra sağlanabilir. Örneğin $American(West)$ kuralın ilk koşulunu karşılar, ama $Weapon$, $Sells$ ve $Hostile$ sonradan oluşabilir. **Rete algoritması**, kısmi eşleşmeleri atmak yerine saklar ve kuralları bir tür veri akışı ağına dönüştürür. Ağın düğümleri koşullardaki literalleri temsil eder; bağlamalar, paylaşılan değişkenler için eşitlik kontrollerinden geçirilir. Bu sayede tekrar tekrar yapılan kısmi eşleştirmeler önlenir.

Rete ve türevleri **production systems (üretim sistemleri)** için temel olmuştur. Bu sistemlerde «production», koşul-eylem kuralını ifade eder. XCON/R1 (McDermott, 1982), Digital Equipment Corporation için bilgisayar bileşenlerini birkaç bin kuralla yapılandırmış ve erken dönem başarılı ticari uzman sistemlerden biri olmuştur. OPS-5 bu teknolojiyi genel amaçlı sunan dillerden biridir. ACT ve SOAR gibi **bilişsel mimariler** üretim kurallarını uzun süreli bellek, çalışma belleğindeki gerçekleri de kısa süreli bellek modeli olarak ele alır. Çalışma döngüsünde koşullar eşleştirilir; uygun eylemler çalışma belleğine gerçek ekleyebilir veya çıkarabilir.

İlgisiz çıkarımlar ve Magic Sets

İleri zincirleme mevcut gerçeklerden **hedefle ilgili olsun olmasın** mümkün sonuçları üretir. Suç örneğine füzelerin fiyatları veya yemek alışkanlıklarıyla ilgili binlerce kural eklersek $Criminal(West)$ sorgusuyla ilgisiz çıkarımlar yapılabilir.

Buna karşı üç seçenek vardır: geri zincirleme kullanmak, yalnız ilgili kuralları seçmek veya **Magic Sets** dönüşümünü uygulamak. Tümdengelimsel veritabanlarında kullanılan Magic Sets, hedef bilgisinden yararlanıp ileri çıkarımda değerlendirilecek bağlamaları sınırlar:

$$Magic(x) \wedge American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x).$$

Bilgi tabanına yalnız $Magic(West)$ eklenirse suç kuralında bütün Amerikalılar yerine West ele alınır. Magic Sets, hedef üzerinden genel geri çıkarıma benzer bir ön işleme ile ileri zincirlemeyi birleştiren **hibrit** bir yaklaşımdır.

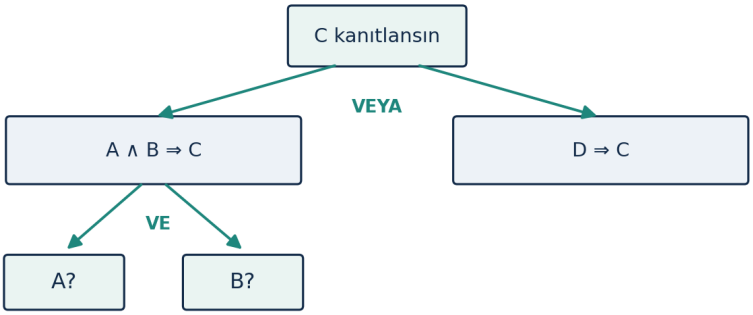
9.4 Geri Zincirleme (Backward Chaining)

Geri zincirleme gerçeklerden değil, **kanıtlanacak sorgudan** başlar. «Bu sonuca hangi kurallar ulaşabilir? Kuralların öncüllerini nasıl kanıtlarım?» sorularını tekrarlayarak bilinen gerçeklere doğru ilerler. Bu yöntem mantıksal programlamanın, özellikle Prolog'un temelidir.

9.4.1 Bir geri zincirleme algoritması

$FOL\text{-}BC\text{-}ASK(KB, goal)$, sağ tarafı hedefle birleştirilebilen $lhs \Rightarrow goal$ biçimli kuralları araştırır. Atomik gerçekler, sol tarafı boş kurallar sayılır. Değişkenli sorgular birden fazla çözüm üretebildiği için yöntem bir **generator (üreteç)** olarak uygulanabilir; örneğin $Person(x)$ hem $\{x/John\}$ hem de $\{x/Richard\}$ yanıtını verebilir.

Geri zincirleme bir **AND/OR aramasıdır**. Bir hedefi kanıtlayan birden fazla alternatif kural varsa aralarında **OR** vardır; seçilen kuralın bütün öncülleri kanıtlanmak zorundaysa aralarında **AND** vardır. $FOL\text{-}BC\text{-}OR$ uygun kuralları seçip hedefle birleştirir, $FOL\text{-}BC\text{-}AND$ ise sol taraftaki bütün öncülleri birikmiş yerine koymayla sırayla kanıtlar.



Şekil 9.6 — Geri zincirleme algoritması

```
FOL-BC-ASK(KB, query):
    FOL-BC-OR(KB, query, {}) sonuçlarını sırayla üret.

FOL-BC-OR(KB, goal, θ):
    Hedefi çözebilecek her (lhs ⇒ rhs) kuralı için:
        Kural değişkenlerini yeniden adlandır.
        θ' = UNIFY(rhs, goal, θ)
        FOL-BC-AND(KB, lhs, θ') tarafından
        üretilen bütün başarılı bağlamaları aktar.

FOL-BC-AND(KB, goals, θ):
    θ = failure ise: başarısız, geri dön
    goals boş ise: θ'yi üret
    değilse:
        first = ilk alt hedef; rest = kalan alt hedefler
        FOL-BC-OR(KB, SUBST(θ, first), θ) ile
        first için her çözümü bul.
        Her çözümü FOL-BC-AND(KB, rest, θ')
        üzerinden sonraki alt hedeflere aktar.
```

Bu biçimiyle algoritma **depth-first search (derinlik öncelikli arama)** yapar; çözümleri biriktirme maliyeti hesaba katılmadığında, izlenen ispat yoluna göre doğrusal bellek kullanır. Ancak tekrar eden durumlar ve sonsuz yollar nedeniyle eksik kalabilir.

Şekil 9.7 — West suç örneğinin geri zincirleme ispat ağacı

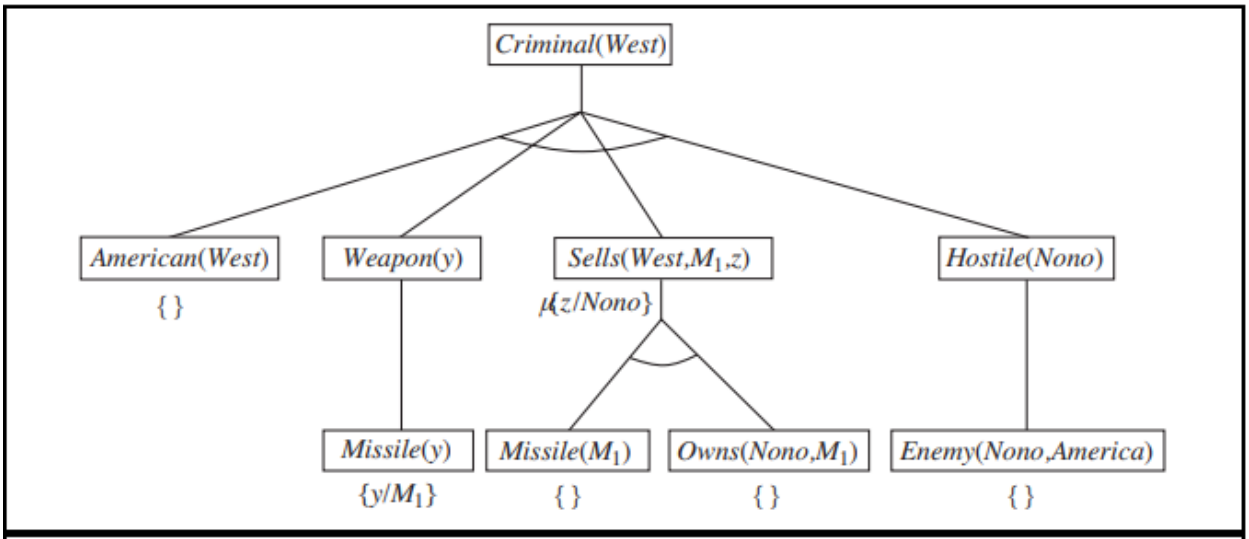


Figure 9.7 Proof tree constructed by backward chaining to prove that West is a criminal. The tree should be read depth first, left to right. To prove $Criminal(West)$, we have to prove the four conjuncts below it. Some of these are in the knowledge base, and others require further backward chaining. Bindings for each successful unification are shown next to the corresponding subgoal. Note that once one subgoal in a conjunction succeeds, its substitution is applied to subsequent subgoals. Thus, by the time FOL-BC-ASK gets to the last conjunct, originally $Hostile(z)$, z is already bound to $Nono$.

En üst hedef $Criminal(West)$'tir. Suç kuralı dört alt hedef oluşturur:

$$American(West), \quad Weapon(y), \quad Sells(West, y, z), \quad Hostile(z).$$

1. $American(West)$ bilgi tabanında vardır; boş bağlama $\{\}$ ile başarı.
2. $Weapon(y)$ için $Missile(y) \Rightarrow Weapon(y)$ kullanılır. $Missile(M_1)$ gerçeğinden $\{y/M_1\}$ alınır.
3. Satış alt hedefi artık $Sells(West, M_1, z)$ olur. Satış kuralının iki koşulu, $Missile(M_1)$ ve $Owns(Nono, M_1)$ gerçeklerince karşılanır. Sonuçta $\{z/Nono\}$ elde edilir.
4. Son alt hedef $Hostile(Nono)$ olur. $Enemy(Nono, America)$ gerçeği ile düşmanlık kuralı bu hedefi kanıtlar.

Bütün alt hedefler başarılıdır; $Criminal(West)$ kanıtlanır. İleri zincirlemede Şekil 9.4 **alttan üste**, geri zincirlemede Şekil 9.7 **üstten alta** okunur. Özellikle bir alt hedefin bağlaması sonraki alt hedeflere uygulanır; $z = Nono$ eşleşmesi bunun açık örneğidir.

9.4.2 Mantıksal programlama (Logic Programming)

Mantıksal programlama, bilginin biçimsel dilde ifade edilmesi ve problemlerin bilgi üzerinde çıkarım yaparak çözülmesi idealini somutlaştırır. Robert Kowalski'nin özeti:

$$\boxed{Algorithm = Logic + Control.}$$

Logic, hangi ilişkilerin geçerli olduğunu; **Control**, kuralların hangi sırayla ve nasıl deneneceğini belirler. En yaygın mantıksal programlama dili **Prolog**'dur. Sembolik işlem, derleyici yazımı, doğal dil ayrıştırma ve hukuk/tıp/finans uzman sistemleri gibi alanlarda kullanılmıştır.

Prolog'da değişkenler büyük, atom/sabitler küçük harfle başlar; `:-` «eğer» anlamında kullanılır; virgül **VE** bağlacını ifade eder. $A \wedge B \Rightarrow C$, `c :- a, b.` olarak yazılır. Suç kuralı:

```
criminal(X) :- american(X), weapon(Y),
               sells(X,Y,Z), hostile(Z).
```

Liste birleştirme — Append ilişkisi

`[E|L]`, baş elemanı E, kalanı L olan listedir. Prolog'da iki listenin art arda eklenmesini aşağıdaki ilişki tanımlar:

```
append([],Y,Y).
append([A|X],Y,[A|Z]) :- append(X,Y,Z).
```

İlk kural «boş liste + Y = Y»; ikinci kural «X ile Y birleştiğinde Z oluyorsa A başlı X ile Y birleştiğinde A başlı Z olur» der. Normal bir fonksiyon iki listeden bir üçüncüsünü üretirken, bu tanım **üç argümanlı ilişki** olduğu için ters yönde de sorgulanabilir:

```
?- append(X,Y,[1,2]).
X = [],    Y = [1,2];
X = [1],   Y = [2];
X = [1,2], Y = [].
```

Bu özellik bildirimsel yaklaşımın yararıdır: hangi ilişkinin geçerli olduğunu yazarız, çözüm yönünü sorguda belirleriz.

Prolog'un uygulamadaki tercihleri

Standart Prolog genellikle kuralları yazılış sırasıyla deneyen **derinlik öncelikli geri zincirleme** uygular. Saf birinci dereceden çıkarımdan ayrıldığı noktalar şunlardır:

- Veritabanı semantiği kullanır; özellikle olumsuzlama ve eşitlik farklı davranır.
- `X is 4+3` aritmetik ifadeyi hesaplayıp X'i 7 yapar; `5 is X+Y` bağlanmamış X ve Y değerlerini denklem çözer gibi bulmaz (çoğu uygulamada hata verir).
- `assert` / `retract` ve giriş/çıkış gibi **yan etkili** hazır işlemler içerir.
- Çoğu geleneksel uygulamada varsayılan unification, occur check'i yapmaz.
- Derinlik öncelikli aramada sonsuz özyinelemeyi otomatik olarak algılamaz; bazı kanıtlanabilir hedefleri bulamayabilir.

Bu tercihler bildirimsel saflıkla yürütme hızı arasında denge kurar.

9.4.3 Mantıksal programların verimli uygulanması

Prolog programları **yorumlanarak** veya **derlenerek** çalıştırılabilir. Yorumlayıcı kabaca FOL-BC-ASK'i çalıştırır, fakat hız için iki önemli veri yapısı kullanır.

Choice point (seçim noktası): Bir hedef birden fazla kuralla veya gerçekle uyuşuyorsa alternatifleri bir yığında saklar. Örneğin `likes(ali,apple).` ve `likes(ali,banana).` gerçeklerinde `likes(ali,X)` sorgusunun ikinci cevabını ararken önceki seçeneğe dönmek için bu yapı kullanılır.

Trail (iz yığını): Mantıksal değişkenlerin hangi değerlere bağlandığını kaydeder. Bir yol başarısız olduğunda önceki choice point'e dönülür ve bu yol sırasında eklenen bağlamalar çözülür. Örneğin `student(X),passed(X)` sorgusunda ilk aday Ali başarısız olursa `X=Ali` geri alınır, ikinci aday Ayşe denenebilir.

Yorumlayıcı her çağrıda indeks bulma, unification ve özyineleme maliyetini taşır. Derleyici programın belli kurallarını önceden bildiğinden her yükleme özel küçük bir çıkarım yordamı oluşturur. Yaygın ara hedef dil **Warren Abstract Machine (WAM)**'dir. Alternatif derleyiciler Prolog'u

Lisp veya C gibi dillere çevirebilir.

Şekil 9.8 — Derlenmiş Append yordamı

```
APPEND(ax, y, az, continuation):
    trail = GLOBAL-TRAIL-POINTER()
    ax = [] ve UNIFY(y, az) ise:
        CALL(continuation)
    RESET-TRAIL(trail)
    a, x, z = NEW-VARIABLE(), NEW-VARIABLE(), NEW-VARIABLE()
    UNIFY(ax, [a|x]) ve UNIFY(az, [a|z]) ise:
        APPEND(x, y, z, continuation)
```

Derlenmiş biçimde `Append` kurallarını bilgi tabanında tekrar tekrar aramak gerekmez. **RESET-TRAIL**, başarısız dallardaki değişken bağlamalarını geri alır. **Continuation**, bir hedef başarılı olduğunda sırada ne yapılacağını paketleyen yordamdır; örneğin sonucu yazdırabilir ya da bir başka kanıtlama adımına geçebilir. Birden fazla çözüm için sıradan tek dönüş (`return`) yeterli olmadığından kullanışlıdır.

Warren ve diğer araştırmacıların derleyicileri Prolog'u belirli standart testlerde C'yle rekabet edebilir hâle getirmiştir. Ayrıca küçük yapay zekâ prototiplerinin kısa Prolog koduyla yazılabilmesi de önemli bir avantajdır.

OR-parallelism: Hedefe uyan alternatif kurallar bağımsız dallar olarak paralel araştırılır. **AND-parallelism**: Bir kuralın farklı öncülleri paralel kanıtlanır; fakat paylaşılan değişkenlerin bağlamaları tutarlı olmalıdır. Bu ikinci durum daha zordur.

9.4.4 Gereksiz çıkarım ve sonsuz döngüler

Prolog'un zayıf noktalarından biri, **derinlik öncelikli arama** ile tekrarlanan durumların ve sonsuz yolların çatışmasıdır. Grafikte yol bulunup bulunmadığını tanımlayan şu programı inceleyelim:

```
path(X,Z) :- link(X,Z).
path(X,Z) :- path(X,Y), link(Y,Z).

link(a,b).
link(b,c).
```

İlk kural doğrudan bağlantıyı, ikinci kural daha uzun yolu ifade eder. $A \to B \to C$ grafiğinde `path(a,c)` doğrudur.

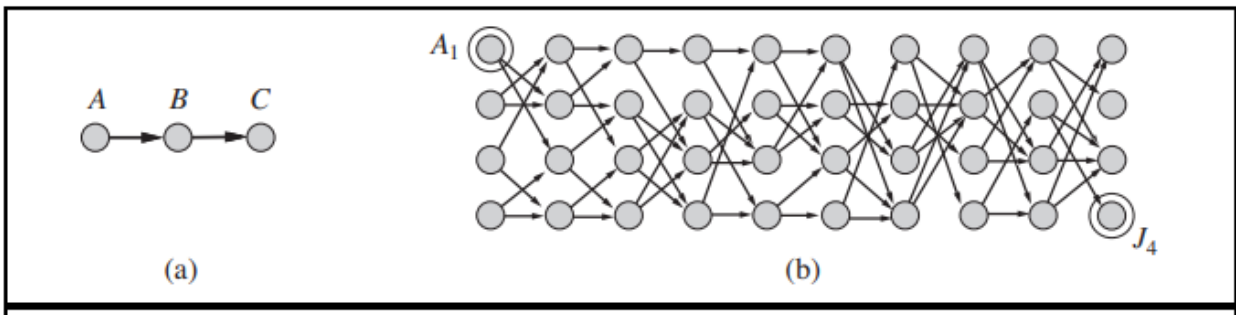


Figure 9.9 (a) Finding a path from A to C can lead Prolog into an infinite loop. (b) A graph in which each node is connected to two random successors in the next layer. Finding a path from A_1 to J_4 requires 877 inferences.

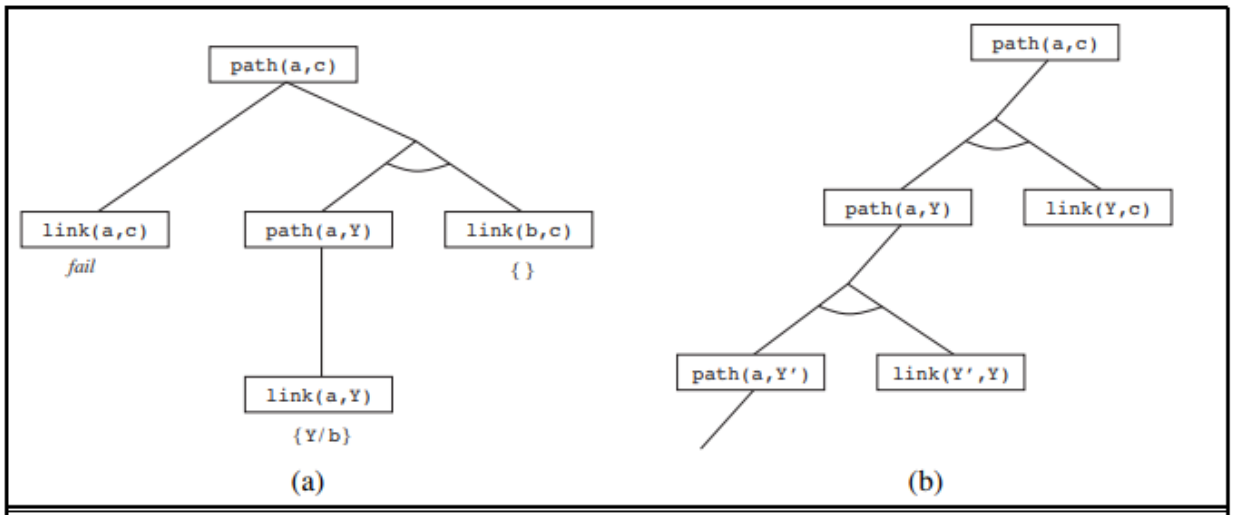


Figure 9.10 (a) Proof that a path exists from A to C. (b) Infinite proof tree generated when the clauses are in the "wrong" order.

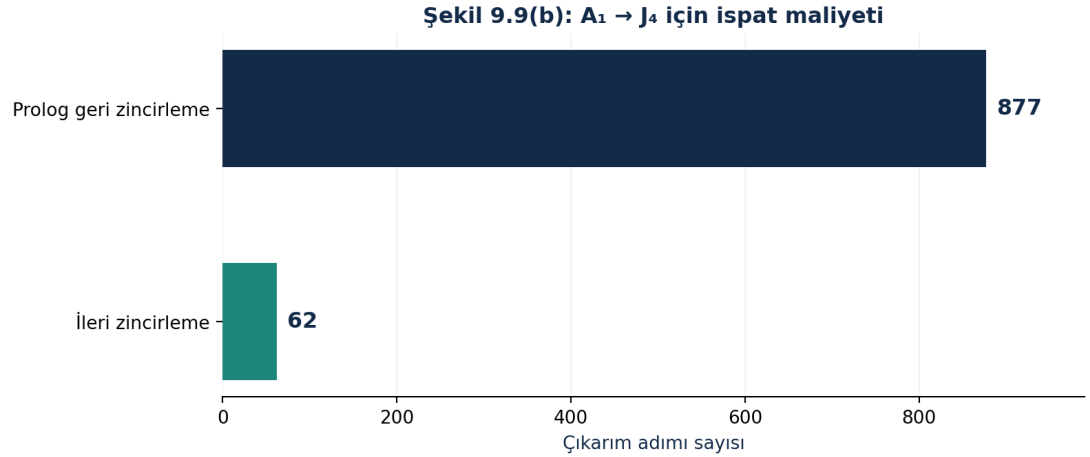
Şekil 9.10(a). İlk kural `link(a,c)` arar ve başarısız olur. İkinci kural `path(a,Y)` ve `link(Y,c)` alt hedeflerini kurar. Doğrudan bağlantıdan $Y = b$ bulunur; $link(b, c)$ de bilindiği için hedef kanıtlanır.

Şimdi kuralları ters sıraya yazalım:

```
path(X,Z) :- path(X,Y), link(Y,Z).
path(X,Z) :- link(X,Z).
```

Şekil 9.10(b). `path(a,c)` önce `path(a,Y)` arar; bu alt hedef yine `path(a,Y')` üretir; sonra `path(a,Y'')`, vb. Sol alt hedef tamamlanamadığından doğrudan bağlantı kuralına geçilemez. Bu nedenle Prolog, bilgi tabanında gerçekten yol olduğu hâlde sonsuz aramaya sapabilir: yürütme stratejisi **tam değildir**. Fonksiyon sembolü bulunmayan bu Datalog örneğinde ileri zincirleme ise yalnız $path(a, b)$, $path(b, c)$ ve $path(a, c)$ gibi gerekli farklı gerçekleri üretip sonlanır.

Şekil 9.9(b)'de A_1'den J_4'e yol aramada kitabın verdiği örnek sayıları:



Prolog, sonuçsuz yolları yeniden araştırarak 877; ileri zincirleme ise 62 çıkarım yapar. **Bu sayılar yalnızca belirtilen örnek grafiğe aittir**; bütün problemlerde ileri zincirlemenin daha hızlı olduğunu göstermez. n düğümlü bir grafikte farklı `path(X,Y)` ground atomları en fazla n^2 adettir.

Dynamic programming (dinamik programlama) benzer alt problemlerin sonuçlarını saklar. Geri zincirlemede bu fikir **memoization** ile uygulanır: daha önce çözülmüş alt hedef tekrar geldiğinde kayıtlı cevap kullanılır. **Tabled logic programming**, geriye doğru hedef odaklı araştırma ile saklanmış alt problem sonuçlarını birleştirir ve sonlu Datalog bilgi tabanlarında tamlık sağlar; ancak sınırsız alanlar için her tür sonsuz davranışı ortadan kaldırmaz.

9.4.5 Prolog'un veritabanı semantiği

Prolog'un veritabanı semantiği iki varsayımla açıklanabilir. **Unique names assumption** farklı sabit/ground terimlerin farklı nesneleri gösterdiğini kabul eder. **Closed-world assumption** ise bilgi tabanından çıkarılamayan atomik bilginin yanlış kabul edilmesidir. Klasik FOL'da söylenmeyen bir ifade «bilinmiyor» olabilir; Prolog'da kapalı dünya davranışı pratik sorgulara daha kolay cevap verir ama ifade gücünü sınırlar.

Şu dört kurs verilsin:

$$Course(CS, 101),\ Course(CS, 102),\ Course(CS, 106),\ Course(EE, 101). \tag{9.11}$$

Prolog semantiğinde CS ve EE, 101/102/106 ayrı isimlerdir; kapalı dünya varsayımıyla bunlardan başka kurs bulunmadığı varsayılır. Sonuç: **tam dört kurs**. Saf FOL'da ise farklı isimler aynı nesneyi gösteriyor olabilir; ayrıca belirtilmemiş kurslar da bulunabilir. Sayı birden sonsuza kadar farklı modellerde değişebilir.

Kurs ilişkisinin FOL'da **completion (tamamlama)** biçimi:

$$\begin{aligned} Course(d, n) \Leftrightarrow\ & (d = CS \wedge n = 101) \vee (d = CS \wedge n = 102) \\ & \vee (d = CS \wedge n = 106) \vee (d = EE \wedge n = 101). \end{aligned} \tag{9.12}$$

Bu, başka kurs çiftlerinin bulunmadığını söyler; listelenenlerin farklı olduğunu söylemek için eşitliğin ayrı biçimde sınırlandırılması da gerekir. Kaynak şu tür bir eşitlik tamamlama şeması gösterir:

$$\begin{aligned} x = y \Leftrightarrow\ & (x = CS \wedge y = CS) \vee (x = EE \wedge y = EE) \\ & \vee (x = 101 \wedge y = 101) \vee (x = 102 \wedge y = 102) \\ & \vee (x = 106 \wedge y = 106). \end{aligned}$$

Uygun olduğu durumda veritabanı semantiğiyle doğrudan akıl yürütmek, bu tür ifadeleri tam FOL ispatlayıcısına dönüştürmekten daha verimlidir.

9.4.6 Kısıt mantığı programlaması (Constraint Logic Programming)

Standart Prolog'un geri izlemeli CSP çözümü sonlu değer alanları için uygundur. Üç renkli harita boyama örneğinde $Diff(Q, SA)$ sorgusunun altı çözümü vardır (renkleri farklı olan altı sıralı çift). Ancak tamsayı/gerçek sayı gibi sınırsız alanlar için yalnız değerleri tek tek denemek yetersizdir; **bounds propagation** ve **linear programming** gibi tekniklere ihtiyaç duyulur.

Kitabın üçgen yüklemi:

```
triangle(X,Y,Z) :-
    X > 0, Y > 0, Z > 0,
    X+Y >= Z, Y+Z >= X, X+Z >= Y.
```

`triangle(3,4,5)` başarılıdır. Fakat `triangle(3,4,Z)` sorgusunda klasik Prolog aritmetik karşılaştırmaları, Z bağlı olmadığı için uygun bir sayısal kısıt çözücü olmadan çalışmaz. **CLP**, değişkeni tek bir değere bağlamak yerine bir kısıtla temsil eder:

$$3 + 4 \geq Z, \quad 4 + Z \geq 3, \quad 3 + Z \geq 4, \quad Z > 0.$$

Birlikte çözünce:

$$\boxed{1 \leq Z \leq 7}.$$



Matematiksel not. Kaynakta zayıf üçgen eşitsizlikleri ($\geq$) yazılı olduğu için 1 ve 7 dahil edilmiştir. Dejenere olmayan geometrik üçgen istenirse sıkı eşitsizliklerle $1 < Z < 7$ gerekir.

CLP sistemleri kısıtları yayabilir, doğrusal programlama kullanabilir, sezgisel koşul sıralaması, backjumping veya cutset conditioning uygulayabilir. Sıradan mantıksal programlama, çözüm kısıtlarının yalnız eşitlik/bağlama olduğu özel CLP durumudur. MRS gibi bazı diller **metarules (üst kurallar)** aracılığıyla örneğin «en az değişken içeren alt hedefi önce dene» türü arama sırası politikalarını programcıya tanımlatır.

9.5 Çözümleme (Resolution)

Üçüncü çıkarım ailesi **Resolution (çözümleme)** üzerine kuruludur. Önerme mantığında çürütme (refutation) yoluyla çözümleme tam bir çıkarım yöntemidir. Birinci dereceden mantık için de kuralı **unification** ile genişletebiliriz. Burada genel tümceleri CNF'ye dönüştürme, çözümleme kuralı, iki ayrıntılı ispat ve tamlık gerekçesi incelenecektir.

9.5.1 Birinci dereceden mantık için CNF

Birinci dereceden çözümleme, ifadelerin **Conjunctive Normal Form (CNF, birleşimsel normal biçim)** olmasını gerektirir: her tümce (*clause*) bir literal ayrışımı ($\vee$), bütün bilgi ise bu tümcelerin birleşimidir ($\wedge$). Literaller değişken içerebilir; bu değişkenler evrensel niceleyicili kabul edilir. Bir tümce alternatif olarak öncülü atomların birleşimi, sonucu atomların ayrışımı olan **implicative/Kowalski form** biçiminde de okunabilir.

Örneğin:

$$American(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)$$

şu CNF tümcesine dönüşür:

$$\boxed{\neg American(x) \lor \neg Weapon(y) \lor \neg Sells(x, y, z) \lor \neg Hostile(z) \lor Criminal(x).}$$

Genel FOL ifadeleri, tatmin edilebilirliği koruyacak bir CNF tümceler kümesine dönüştürülebilir. Bu nedenle çelişkiyle ispatta CNF güvenle kullanılabilir. Dönüşümün en önemli yeni adımı varoluşsal niceleyicilerin **Skolemleştirme** yoluyla kaldırılmasıdır.

«Bütün hayvanları seven herkes birisi tarafından sevilir» örneği

Kaynağın ifadesini kapsamları açık göstererek yazalım (tekrarlanan niceleyici adı karışmasın diye ikinci tanık z olsun):

$$\forall x \Big[(\forall y \, [Animal(y) \Rightarrow Loves(x, y)]) \Rightarrow (\exists z \, Loves(z, x)) \Big].$$

1. Koşullu ifadeleri kaldır. $P \Rightarrow Q \equiv \neg P \lor Q$:

$$\forall x \Big[\neg \forall y \, (\neg Animal(y) \lor Loves(x, y)) \lor \exists z \, Loves(z, x) \Big].$$

2. Olumsuzluğu içeri taşı. De Morgan kuralları ile niceleyici karşıtlıklarını kullanırız:

$$\neg \forall v \, P \equiv \exists v \, \neg P, \qquad \neg \exists v \, P \equiv \forall v \, \neg P.$$

İfade sırasıyla:

$$\forall x [\exists y \, \neg (\neg Animal(y) \lor Loves(x, y)) \lor \exists z \, Loves(z, x)]$$

ve:

$$\boxed{\forall x [\exists y \, (Animal(y) \land \neg Loves(x, y)) \lor \exists z \, Loves(z, x)]}$$

olur. Anlamı: «Her x için, ya x'in sevmediği bir hayvan vardır ya da x'i seven birisi vardır.» Bu, ilk koşullu ifadenin eşdeğer başka söylenişidir.

3. Değişkenleri ayır (standardize apart). Birbirinden bağımsız varoluşsal kapsamlar aynı değişken adını kullanmışsa farklı adlar verilir. Biz baştan y ve z seçtik. Örneğin $(\exists x P(x)) \lor (\exists x Q(x))$ ifadesinde ikinci x, y olarak yeniden adlandırılır.

4. Skolemization. Basit $\exists v \, P(v)$ için yeni sabit A kullanılabilir; ama içteki varoluşsal tanıklar dış $\forall x$ değişkenine **bağlıdır**. Bu nedenle tek ve sabit A/B yazmak yanlıştır: herkesin aynı hayvanı sevmediğini veya aynı kişi tarafından sevildiğini öne sürmüş oluruz. Varoluşsal y ve z için sırasıyla **Skolem fonksiyonları** $F(x)$ ve $G(x)$ gerekir:

$$\boxed{\forall x [(Animal(F(x)) \land \neg Loves(x, F(x))) \lor Loves(G(x), x)]}.$$

Skolem fonksiyonunun argümanları, ilgili varoluşsal niceleyicinin kapsamında bulunduğu evrensel değişkenlerdir. Örneğin $\forall x \exists y \, R(x, y)$ ifadesi $R(x, F(x))$ olur; başında evrensel niceleyici olmayan $\exists y \, R(A, y)$ ifadesinde ise yeni sabit yeterlidir. Skolemleştirme **tatmin edile-**

bilirlik eşdeğerliğini korur; her zaman sözdizimsel/mantıksal eşdeğerlik kurmaz.

5. Evrensel niceleyicileri kaldır. Artık kalan bütün değişkenler evrensel kabul edildiğinden yazılmayabilir:

$$(Animal(F(x)) \land \neg Loves(x, F(x))) \lor Loves(G(x), x).$$

6. $\lor$ işlemini $\land$ üzerine dağıt. $(A \land B) \lor C \equiv (A \lor C) \land (B \lor C)$:

$$\boxed{\begin{array}{r}[Animal(F(x)) \lor Loves(G(x), x)] \land \\ [\neg Loves(x, F(x)) \lor Loves(G(x), x)].\end{array}}$$

Sonuç iki CNF tümcesidir. $F(x)$, x'in sevmediği olası hayvanı; $G(x)$, x'i seven olası kişiyi temsil eder. İnsan için karmaşık görünen bu dönüşüm mekanik olarak yapılabilir.

Kaynak gösterimi hakkında not. Gönderilen metnin Skolemleştirme ve sonrasındaki birkaç satırında $G(z)$ yazılı görünüyor; ancak Skolem fonksiyonu dıştaki evrensel x'e bağlı olmalıdır. Aynı bölümün sonraki ispatı da $G(x)$ kullanır. Tutarlı sonuç burada $G(x)$ olarak verilmiştir.

Dönüşüm adımı	Kısa kural
Koşulluyu kaldır	$A \Rightarrow B \equiv \neg A \lor B$
Olumsuzluğu içeri taşı	$\neg(A \land B) \equiv \neg A \lor \neg B$
Niceleyici olumsuzluğu	$\neg \forall x P \equiv \exists x \neg P$
Değişkenleri standartlaştır	Bağımsız kapsamlar için ayrı değişken adı
Skolemleştir	$\exists$ tanığını sabit/fonksiyonla değiştir
Evrenselleri gizle	Kalan değişkenler örtük evrenseldir
Dağıt	$A \lor (B \land C) \equiv (A \lor B) \land (A \lor C)$

9.5.2 Resolution çıkarım kuralı

Birinci dereceden çözümleme, önerme mantığındaki kuralın **lifted** biçimidir. İki tümce önce değişken çakışmasını önlemek için ayrı standartlaştırılır. Tamamlayıcı literaller, karşıt işaretli olmanın yanında **birleştirilebilir** olmalıdır.

İki tümce:

$$L_1 \lor \cdots \lor L_k, \qquad M_1 \lor \cdots \lor M_n$$

ve $UNIFY(L_i, \neg M_j) = \theta$ olsun. O zaman L_i ve M_j elenir; geri kalan literallerin ayrışımına θ uygulanarak **resolvent** elde edilir. Önerme mantığındaki basit örnek $A \lor B$ ve $\neg B \lor C$'den $A \lor C$ elde edilmesidir.

Birinci dereceden örnekte:

$$Animal(F(x)) \lor Loves(G(x), x)$$

ve:

$$\neg Loves(u, v) \lor \neg Kills(u, v)$$

ifadeleri için tamamlayıcı literallerin unifier'ı:

$$\theta = \{u/G(x), v/x\}$$

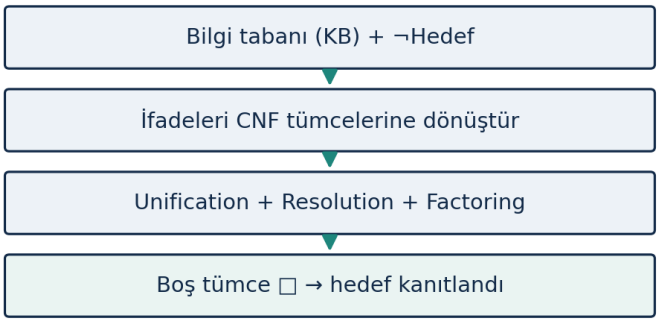
olur. Çözümleme sonucu:

$$\boxed{Animal(F(x)) \vee \neg Kills(G(x), x).}$$

Binary resolution, tek bir adımda iki tamamlayıcı literal çifti üzerinden çözümleme yapar. Tek başına genel tamlık sağlamaz. **Factoring** ise bir tümcedeki aynı veya birleştirilebilir yinelenen literalleri teke indirir. Örneğin $P(x) \vee P(Ali)$ ifadesine $\{x/Ali\}$ uygulanıp tekrar çıkarılırsa $P(Ali)$ kalır. **Binary resolution + factoring** birlikte genel bir çürütme-tam yöntemi verir.

9.5.3 Örnek çözümleme ispatları

Resolution, $KB \models \alpha$ sonucunu göstermek için $KB \wedge \neg\alpha$ ifadesinin **tatmin edilemez** olduğunu ispatlar. Bunu yapmanın işareti **empty clause (boş tümce)**, $\square$, elde etmektir. Boş tümce bütün literallerin elendiği çelişki durumudur.



Birinci ispat: Albay West suçlu mudur? (Şekil 9.11)

Suç örneğinin CNF tümceleri:

$$\begin{aligned}
&C_1 : \neg American(x) \vee \neg Weapon(y) \vee \neg Sells(x, y, z) \vee \\
&\quad \neg Hostile(z) \vee Criminal(x), \\
&C_2 : \neg Missile(x) \vee \neg Owns(Nono, x) \vee Sells(West, x, Nono), \\
&C_3 : \neg Enemy(x, America) \vee Hostile(x), \\
&C_4 : \neg Missile(x) \vee Weapon(x), \\
&C_5 : Owns(Nono, M_1), \qquad C_6 : \ Missile(M_1), \\
&C_7 : American(West), \qquad C_8 : \ Enemy(Nono, America).
\end{aligned}$$

Kanıtlanacak hedef $Criminal(West)$ olduğundan $\neg Criminal(West)$ eklenir.

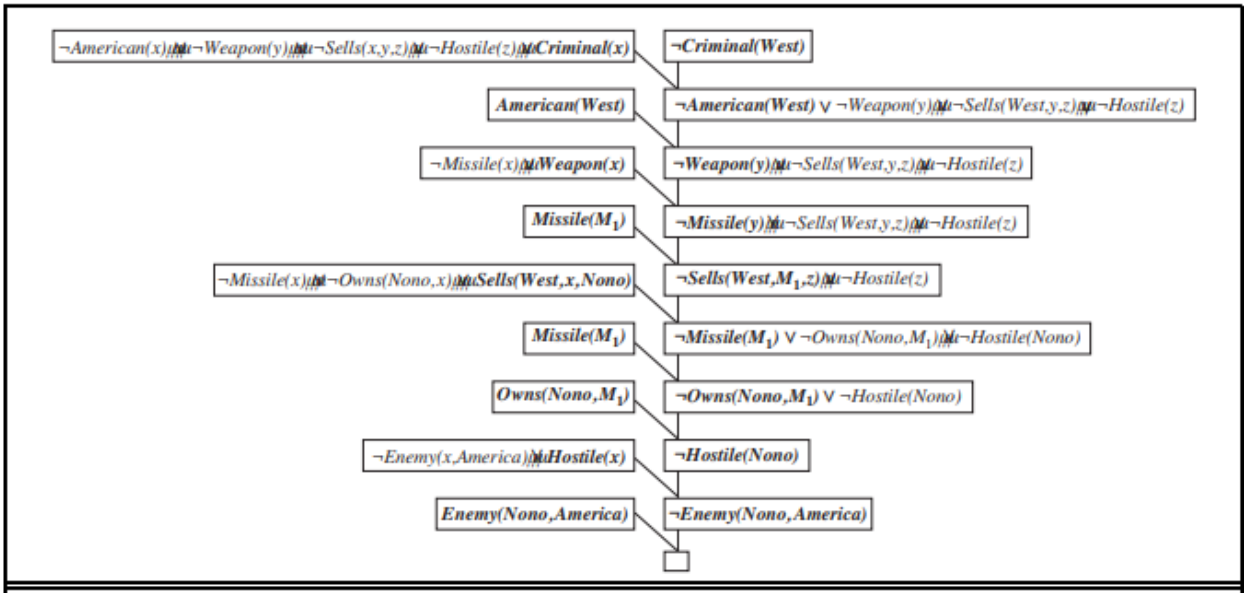


Figure 9.11 A resolution proof that West is a criminal. At each step, the literals that unify are in bold.

Şekil 9.11'in adım adım okunması (ana omurga).

1. C_1 ile $\neg Criminal(West)$ çözümle: $\{x/West\}$; sonuç $\neg American(West) \vee \neg Weapon(y) \vee \neg Sells(West, y, z) \vee \neg Hostile(z)$.
2. $American(West)$ gerçeğiyle $\neg American(West)$ elenir; geriye $\neg Weapon(y) \vee \neg Sells(West, y, z) \vee \neg Hostile(z)$ kalır.
3. C_4 ile $Weapon(y)$ ve $\neg Weapon(y)$ çözümle: $\neg Missile(y) \vee \neg Sells(West, y, z) \vee \neg Hostile(z)$.
4. $Missile(M_1)$ gerçeğiyle $\{y/M_1\}$ uygula: $\neg Sells(West, M_1, z) \vee \neg Hostile(z)$.
5. Satış kuralı C_2 ile $\{x/M_1, z/Nono\}$ uygula: $\neg Missile(M_1) \vee \neg Owns(Nono, M_1) \vee \neg Hostile(Nono)$.
6. $Missile(M_1)$ ile füze olumsuzlamasını ele: $\neg Owns(Nono, M_1) \vee \neg Hostile(Nono)$.
7. $Owns(Nono, M_1)$ ile sahiplik olumsuzlamasını ele: $\neg Hostile(Nono)$.
8. C_3 ile $\{x/Nono\}$ uygula: $\neg Enemy(Nono, America)$.
9. $Enemy(Nono, America)$ gerçeğiyle son karşıt literali ele: $\square$.

Böylece $KB \wedge \neg Criminal(West)$ tutarsız olduğundan:

$$\boxed{KB \models Criminal(West).}$$

Şeklin tek bir ana omurga (**spine**) içermesi, Horn tümceleriyle yapılan ve sol baştaki hedefi sırayla çözen geri zincirlemeye benzer. Aslında **backward chaining**, uygun bir kontrol stratejisiyle seçilen özel türde Resolution olarak görülebilir.

İkinci ispat: Curiosity Tuna adlı kediyi öldürdü mü? (Şekil 9.12)

Verilenler:

- Bütün hayvanları seven herkes birisi tarafından sevilir.
- Bir hayvanı öldüren kişi hiç kimse tarafından sevilmez.
- Jack bütün hayvanları sever.
- Tuna'yı Jack veya Curiosity öldürmüştür.

- Tuna bir kedidir ve bütün kediler hayvandır.

Soru: $Kills(Curiosity, Tuna)$ **doğru mu?**

Kaynağın FOL ifadeleri (niceleyici kapsamları açıklaştırılmıştır):

$$
\begin{aligned}
A &: \forall x \Big[(\forall y \, [Animal(y) \Rightarrow Loves(x, y)]) \Rightarrow \exists z \, Loves(z, x) \Big], \\
B &: \forall x \Big[(\exists z \, [Animal(z) \wedge Kills(x, z)]) \Rightarrow \forall y \, \neg Loves(y, x) \Big], \\
C &: \forall x \, [Animal(x) \Rightarrow Loves(Jack, x)], \\
D &: Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna), \\
E &: Cat(Tuna), \\
F &: \forall x \, [Cat(x) \Rightarrow Animal(x)], \\
\neg G &: \neg Kills(Curiosity, Tuna).
\end{aligned}
$$

CNF dönüşümünden sonra:

$$
\begin{aligned}
A_1 &: Animal(F(x)) \vee Loves(G(x), x), \\
A_2 &: \neg Loves(x, F(x)) \vee Loves(G(x), x), \\
B &: \neg Loves(y, x) \vee \neg Animal(z) \vee \neg Kills(x, z), \\
C &: \neg Animal(x) \vee Loves(Jack, x), \\
D &: Kills(Jack, Tuna) \vee Kills(Curiosity, Tuna), \\
E &: Cat(Tuna), \\
F &: \neg Cat(x) \vee Animal(x), \\
\neg G &: \neg Kills(Curiosity, Tuna).
\end{aligned}
$$

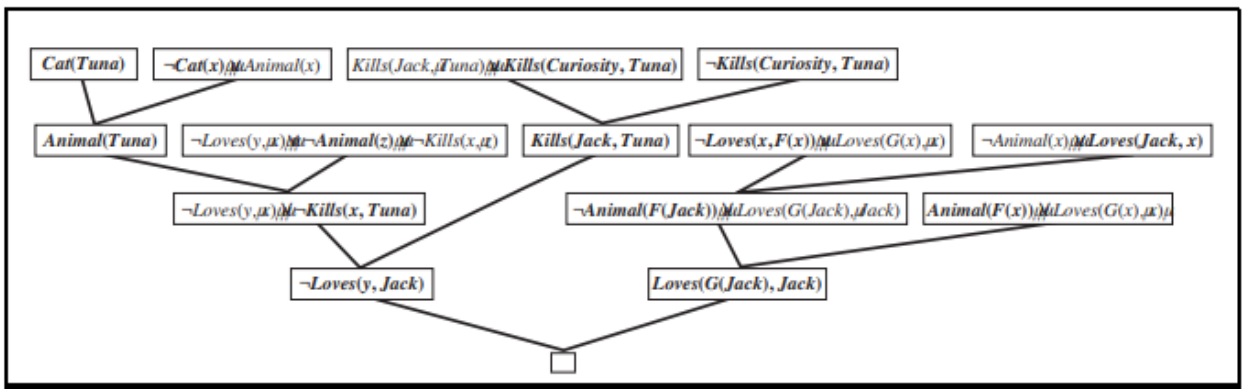


Figure 9.12 A resolution proof that Curiosity killed the cat. Notice the use of factoring in the derivation of the clause $Loves(G(Jack), Jack)$. Notice also in the upper right, the unification of $Loves(x, F(x))$ and $Loves(Jack, x)$ can only succeed after the variables have been standardized apart.

İspatın Türkçe açıklaması, sol kol: Curiosity'nin Tuna'yı öldürmediğini varsayalım. D tümcesinde Jack veya Curiosity'nin öldürdüğü bilindiğinden $Kills(Jack, Tuna)$ çıkar. $Cat(Tuna)$ ve $Cat(x) \Rightarrow Animal(x)$ ile $Animal(Tuna)$ elde edilir. Hayvan öldürenlerin sevilmediği B tümcesinde $x = Jack, z = Tuna$ kullanınca:

$$\boxed{\neg Loves(y, Jack)}$$

elde edilir: «Hiç kimse Jack'i sevmez.»

İspatın Türkçe açıklaması, sağ kol: A1 ve C tümcelerinden, değişkenleri ayrı adlandırıp çözümleme yaparak:

$$Loves(G(x), x) \vee Loves(Jack, F(x))$$

buluruz. Bunu A2'nin standartlaştırılmış $\neg Loves(v, F(v)) \vee Loves(G(v), v)$ biçimiyle çözümleyince $v = Jack, x = Jack$ olur ve:

$$Loves(G(Jack), Jack) \vee Loves(G(Jack), Jack)$$

üretilir. **Factoring** ile yinelenen literal teke indirilir:

$$\boxed{Loves(G(Jack), Jack)}.$$

Yani Jack'i seven en az birisi vardır. Son olarak $\neg Loves(y, Jack)$ ile $\{y/G(Jack)\}$ altında çözümleme boş tümce üretir: $\square$. Başlangıçtaki $\neg Kills(Curiosity, Tuna)$ varsayımı çelişkili olduğundan:

$$\boxed{Kills(Curiosity, Tuna)}.$$

Şekilde sağ üstteki iki Loves literalinin eşleşmesi, farklı tümcelerin değişkenleri önce **standardizing apart** ile ayrıldığında doğru çalışır. Ayrıca «Jack'i seven biri vardır» sonucunda iki aynı literal ortaya çıktığı için **factoring** gerekir. Bu ispat, definite clause olmayan ifadelerin de ele alınabildiğini gösterir.

Soru «Kediyi kim öldürdü?» olursa

Hedef $\exists w\, Kills(w, Tuna)$, olumsuz hedef ise $\neg Kills(w, Tuna)$ biçimine gelir. Şekil 9.12'nin ispatı $\{w/Curiosity\}$ bağlamasını verebilir. Ancak Resolution bazen **nonconstructive (yapıcı olmayan)** varoluş ispatları üretir: aynı sorgu değişkeni bir adımda Jack, diğer adımda Curiosity değerini alabilir; bu ispat sadece «birisi öldürdü» der, tek bir cevap vermez.

Buna çözüm olarak sorgu değişkenlerinin tek kanıt içinde bir kez bağlanması veya özel bir **answer literal** eklenmesi önerilir:

$$\boxed{\neg Kills(w, Tuna) \vee Answer(w)}.$$

Bu yöntemde $Answer(Curiosity)$ tekil tümcesi belirli bir cevap sunar. $Answer(Jack) \vee Answer(Curiosity)$ ise tek bir tanık belirlemediğinden aynı türde cevap sayılmaz.

9.5.4 Resolution'ın tamlığı (Completeness of Resolution)

Burada ispatlanacak özellik **refutation-completeness (çürütme tamlığı)**dır: S tümceler kümesi tatmin edilemezse Resolution sonlu sayıda uygun adımla boş tümceyi üretebilir. Bu, her olumsuz sorguda yöntemin sonlanacağı anlamına gelmez.

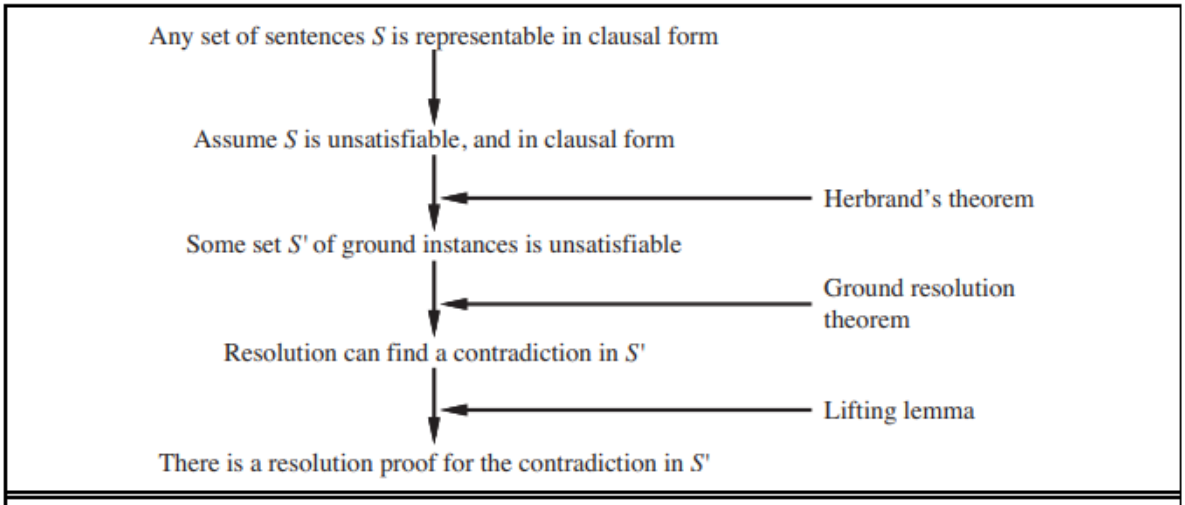


Figure 9.13 Structure of a completeness proof for resolution.

Şekil 9.13'ün yapısı:

1. Her FOL ifade kümesi CNF tümcelerine dönüştürülebilir (eşitlik özel durumları şimdilik hariç).
2. S'nin tatmin edilemez olduğu varsayılır.
3. **Herbrand teoremi:** S'nin ground örneklerinden oluşan tatmin edilemeyen sonlu bir S' alt kümesi vardır.
4. **Ground resolution teoremi:** Önerme mantığındaki Resolution, S' içinden boş tümceyi türetir.
5. **Lifting lemma:** Ground örnekler üzerindeki çözümleme adımlarının karşılıkları doğrudan birinci dereceden tümceler üzerinde de vardır.
6. O hâlde özgün S üzerinde de boş tümceye ulaşılabilir.

Herbrand evreni, doyurma ve Herbrand tabanı

S kümesindeki sabit ve fonksiyon sembollerinden oluşturulabilen bütün ground terimler **Herbrand universe** H_S kümesini oluşturur. Sabit yoksa yeni bir A sabiti eklenir. Örnek tümce:

$$\neg P(x, F(x, A)) \vee \neg Q(x, A) \vee R(x, B).$$

Herbrand evreninin başlangıcı:

$$H_S = \{A, B, F(A, A), F(A, B), F(B, A), F(B, B), F(A, F(A, A)), ...\}.$$

Saturation (doyurma), S'deki değişkenleri seçilen ground terimler kümesinin bütün uygun elemanlarıyla değiştirmekten elde edilen ground tümceler kümesidir. Kaynak, S'nin tüm Herbrand evreniyle doyurulmasına **Herbrand base** $H_S(S)$ adını verir. Örneğin x=A için:

$$\neg P(A, F(A, A)) \vee \neg Q(A, A) \vee R(A, B);$$

x=B için:

$$\neg P(B, F(B, A)) \vee \neg Q(B, A) \vee R(B, B).$$

Terminoloji notu: Başka mantık kitapları «Herbrand base» terimini, tüm ground atomların kümesi için kullanır. Burada kaynak metindeki «tümcelerin doyurulması» kullanımını izledik.

Herbrand teoremi: S tatmin edilemezse $H_S(S)$ içinde tatmin edilemeyen sonlu bir alt küme vardır. Bu, sonsuz örnek alanından sonlu bir çelişki kanıtına ulaşmayı mümkün kılar.

Lifting Lemma

İki bağımsız değişken kümesine sahip tümce C_1, C_2 ve bunların ground örnekleri C_1', C_2' olsun. Eğer C' ground tümcelerin çözümleme sonucuysa, özgünlerden elde edilen bir resolvent C bulunur ve C' C'nin ground örneğidir. Böylece ground düzeyindeki çözümleme **birinci dereceden düzeye yükseltilir**.

Kaynağın örneği:

$$C_1 = \neg P(x, F(x, A)) \vee \neg Q(x, A) \vee R(x, B),$$
$$C_2 = \neg N(G(y), z) \vee P(H(y), z).$$

Tamamlayıcı P-literalinin unifier'ı:

$$\theta = \{x/H(y),\ z/F(H(y), A)\}.$$

Böylece doğrudan genel sonuç:

$$\boxed{\begin{aligned} C = {} & \neg N(G(y), F(H(y), A)) \\ & \vee \neg Q(H(y), A) \vee R(H(y), B). \end{aligned}}$$

elde edilir. $y = B$ koyulursa kaynakta verilen ground C' üretilir:

$$\begin{aligned} C' = {} & \neg N(G(B), F(H(B), A)) \\ & \vee \neg Q(H(B), A) \vee R(H(B), B). \end{aligned}$$

Ground ispatın her adımı bu biçimde yükseltilebilir. Dolayısıyla ground kapanışta bulunan boş tümce, genel birinci dereceden kapanışta da bulunur. Böylece Resolution'ın çürütme açısından tam olduğu kanıtlanır. Bu kuramda **Robinson'ın unification ve MGU** kavramlarını geliştirmesi kritik rol oynamıştır.

Yan kutu — Gödel'in Eksiklik Teoremi

Kaynağın yan açıklaması, Resolution tamlığı ile aritmetikteki **Gödel eksikliği** arasındaki görünürdeki gerilime değinir. Doğal sayıları 0, ardıl fonksiyonu S (ör. $S(0) = 1$) ve aritmetik işlemlerle gösterebiliriz. Gödel, ifadeleri ve ispatları doğal sayılarla kodlayabilen (Gödel numaralandırması) yeterince güçlü sistemlerde, kendi ispatlanamazlığını ifade eden cümlelerin kurulabileceğini gösterdi. Etkili biçimde aksiyomlaştırılmış, tutarlı ve yeterince güçlü aritmetik kuramları, doğal sayılarda doğru olan bütün cümleleri ispatlayamaz.

Bu durum Resolution tamlığıyla çelişmez: **Bir cümlenin amaçlanan doğal sayılar modelinde doğru olması**, onun seçilen aksiyomlardan mantıksal olarak çıkmasıyla aynı şey değildir. Resolution tamlığı, verilen aksiyomlardan mantıksal olarak çıkan sonuçlarla ilgilidir.

9.5.5 Eşitlik (Equality)

Şimdiye kadarki yöntemler $x = y$ ifadelerini özel olarak işlemiyordu. Eşitlik için üç yaklaşım vardır: eşitliği aksiyomlaştırmak; özel çıkarım kuralları eklemek; veya birleştirmeyi eşitlik teorisine göre genişletmek.

Eşitlik aksiyomları

Eşitliğin **yansımalı, simetrik, geçişli** olması ve eşitlerin yüklem/fonksiyonlarda birbirlerinin yerine konabilmesi gerekir:

$$\begin{aligned}
&\forall x\ (x = x),\\
&\forall x, y\ (x = y \Rightarrow y = x),\\
&\forall x, y, z\ [(x = y \wedge y = z) \Rightarrow x = z],\\
&\forall x, y\ [x = y \Rightarrow (P(x) \Leftrightarrow P(y))],\\
&\forall w, x, y, z\ [(w = y \wedge x = z) \Rightarrow F(w, x) = F(y, z)].
\end{aligned}$$

Her yüklem/fonksiyon için gerekli yer-değiştirme aksiyomları eklenirse normal Resolution ile eşitlik çıkarımları yapılabilir. Ancak ilgisiz çok sayıda türetim doğabileceğinden daha özel yöntemler tercih edilir.

Demodulation (yönlü yeniden yazma)

$u = v$ biçiminde tekil eşitlik ve içinde u ile birleştirilebilen bir terim bulunan tümce verildiğinde uygun terimler v'ye **tek yönde** dönüştürülür. Örneğin $x + 0 = x$ ve $x \times 1 = x$ sadeleştirme kuralları gibi çalışır. Kaynak örneği:

$$Father(Father(x)) = PaternalGrandfather(x)$$

ve:

$$Birthdate(Father(Father(Bella)), 1926)$$

ifadelerinden:

$$\boxed{Birthdate(PaternalGrandfather(Bella), 1926)}$$

üretilir. SUBST birleştiriciyi uygular; SUB belirli terimi başka bir terimle yeniden yazar. Demodulation yönlüdür: kuralın sol yanı sağ yanla değiştirilir.

Paramodulation

Eşitlik birim tümceyle sınırlı olmayıp ayrışımın içinde de bulunabilir. Örnek:

$$P(F(x, B), x) \vee Q(x)$$

ve:

$$F(A, y) = y \vee R(y).$$

$F(A, y)$ ile $F(x, B)$ için $\theta = \{x/A, y/B\}$ bulunur. Bu bağlamayla eşitlik $F(A, B) = B$ olur; birinci tümcedeki ilgili terim B ile değiştirilir, kalan literaller korunur:

$$\boxed{P(B, A) \vee Q(A) \vee R(B).}$$

Paramodulation, eşitliği içeren birinci dereceden mantıkta tam çıkarım prosedürlerinin kurulmasını sağlar.

Equational Unification

Üçüncü yöntemde iki terim, normal yapısal eşleşmeyle değil, **verilen eşitlik teorisine göre** eşit oldukları kanıtlanabiliyorsa birleştirilebilir kabul edilir. Örneğin $1 + 2$ ile $2 + 1$ normal sözdizimsel eşleştirmede farklıdır; ancak toplamanın değişme kuralı $a + b = b + a$ biliniyorsa boş $\theta = \{\}$ ile eşdeğer kabul edilebilir. Eşitlik teorisine özel algoritmalar (değişme, birleşme vb.) her aksiyomu açıkça tekrar tekrar çözümlemekten daha verimli olabilir. Bu sistemler CLP'ye yakındır.

9.5.6 Çözümleme stratejileri (Resolution Strategies)

Resolution sonunda var olan ispatı bulabilir; fakat her olası tümce çiftini denemek çok pahalı olabilir. Aşağıdaki stratejiler aramayı sınırlandırır veya daha umut verici adımlara öncelik verir.

Unit Preference / Unit Resolution

Unit clause yalnız bir literal içerir. **Unit Preference** böyle tümcelerle çözümlemeyi öncelemeyi önerir; çünkü çoğu zaman daha kısa yeni tümce üretir. Örneğin:

$$P, \qquad \neg P \vee \neg Q \vee R \quad \Rightarrow \quad \neg Q \vee R.$$

Unit Resolution ise her adımda bir tarafın mutlaka unit clause olmasını *zorunlu* kılar. Genel durumda eksiktir, Horn tümceleri için tamdır ve Horn üzerinde ileri zincirlemeye benzer. Tarihsel **OTTER** ispatlayıcısı, kısa/«hafif» tümcelere daha yüksek öncelik veren ağırlıklı en iyi-öncelikli arama kullanır; bu, unit preference fikrinin genelleştirilmesidir.

Set of Support (destek kümesi)

Her çözümleme adımının en az bir tarafının seçilmiş bir **destek kümesinden** gelmesi istenir; yeni resolvent de bu kümeye katılır. Küme bilgi tabanından küçükse arama uzayı önemli ölçüde daralır. Eğer destek kümesinin **dışında** kalan tümceler birlikte tatmin edilebilir ise yöntem tamdır. Tutarlı bir KB'de olumsuzlanmış sorguyu destek kümesine koymak yaygın seçimdir; ayrıca insanlar için okunabilir, hedef odaklı ağaçlar oluşturabilir.

Input Resolution ve Linear Resolution

Input Resolution her çözümleme adımında, iki tümceden en az birinin özgün girdiler (KB veya ilk sorgu) arasında bulunmasını ister. Şekil 9.11'deki tek omurgalı West ispatı bunun örneğidir. Horn bilgi tabanları için tam, genel durum için eksiktir.

Linear Resolution biraz daha esnektir: iki tümceden biri özgün KB'de bulunuyor **veya** ispat ağacında diğerinin atası ise birlikte çözümlenebilirler. Bu strateji tamdır.

Subsumption (kapsama yoluyla eleme)

Bilgi tabanında daha genel bir tümce varsa onun mantıksal olarak kapsadığı daha özel tümceler gereksizdir. Örneğin $P(x)$ bütün nesneler için geçerliyse yeni $P(A)$ tümcesini eklemeye gerek yoktur; $P(A) \vee Q(B)$ de ek bilgi sağlamaz. Bu tür ifadelerin silinmesi bilgi tabanını ve çözümleme arama uzayını küçültür.

Strateji	Etkisi	Genel tamlık
Unit Preference	Birim tümceyi önce dene	Sadece sıralama tercihidir
Unit Resolution	Bir taraf birim olmak zorunda	Horn için tam, genelde eksik
Set of Support	Özel hedef kümesinden en az bir taraf kullan	Dış küme tatmin edilebilirse tam
Input Resolution	Her adımda özgün girdi tümcesi kullan	Horn için tam, genelde eksik
Linear Resolution	Girdi veya ata tümce kullanılabilir	Tam
Subsumption	Gereksiz özel tümceleri at	Aramayı sadeleştirir

Resolution teorem ispatlayıcılarının uygulamaları

Teorem ispatlayıcıları sadece AI'da değil, **donanım ve yazılım doğrulama/sentezinde** de kullanılmıştır. Donanımda sinyaller ve devre elemanlarının etkileşimi aksiyomlarla modellenebilir; bazı tarihsel çalışmalar CPU zamanlama özelliklerinin doğrulanmasını, AURA ile devre tasarımını göstermiştir. Yazılımda program komutlarının önkoşul ve etkileri biçimsel olarak temsil edilir. **Deductive synthesis (tümdengelimsel sentez)**, «belirli bir şartnameyi sağlayan program p vardır» önermesini yapıcı olarak ispatlayıp program türetme fikridir. Kitap, tam otomatik genel-amaçlı sentezin sınırlı oluşunu ve insan yönlendirmeli/özel amaçlı sentezdeki başarıları tartışır; SPIN, Remote Agent, RSA ve Boyer–Moore ile ilişkili biçimsel doğrulama çalışmalarını tarihsel örnekler olarak anmaktadır.

Ek A — Bütün konuları birleştiren çözümlü örnekler

Bu bölümdeki örnekler, **kitap metnine ilave öğretici çalışma soruları** olarak hazırlanmıştır. Kullanılan çıkarım kurallarının sırasını özellikle görünür kılmak amaçlanmıştır.

A.1 Evrensel ve varoluşsal niceleyici

Soru. Bilgi tabanında $\forall x\,[Teacher(x) \Rightarrow Employee(x)]$, $Teacher(Ayse)$ ve $\exists z\,[Teaches(z, AI)]$ veriliyor. Hangi sonuçlar çıkar?

Çözüm. Evrensel ifadede $x = Ayse$ koyarız: $Teacher(Ayse) \Rightarrow Employee(Ayse)$. $Teacher(Ayse)$ ile Modus Ponens sonucu $Employee(Ayse)$ çıkar. Varoluşsal ifadeye EI uygulayıp daha önce kullanılmamış T_1 sabiti üretiriz: $Teaches(T_1, AI)$. Ancak $T_1 = Ayse$ diyemeyiz; varoluşsal cümle derse giren kişinin kimliğini belirtmiyor. **Kritik fark:** UI bilinen sabite genel kural uygular; EI kimliği belirsiz tanığı yeni isimle adlandırır.

A.2 Genelleştirilmiş Modus Ponens ve bağlama aktarımı

Soru. $Registered(x, c) \wedge CompletedPrerequisite(x, c) \Rightarrow Allowed(x, c)$ kuralı, $Registered(Mehmet, AI101)$ ve $CompletedPrerequisite(Mehmet, AI101)$ gerçekleri verilmiş olsun. Sonuç nedir?

Çözüm. İki öncülün aynı kişiyi ve aynı dersi ifade etmesi gerekir. $\theta = \{x/Mehmet, c/AI101\}$ her iki koşulu da eşleştirir. Sonuç $Allowed(Mehmet, AI101)$ olur. İlk koşulda Mehmet, ikinci koşulda Ayşe olsaydı aynı x üzerinde çakışma nedeniyle kural uygulanamazdı.

A.3 Birleştirme testi: altı farklı durum

Aşağıdaki tablodaki her satır bağımsız bir UNIFY çağrısıdır.

İfade 1	İfade 2	Sonuç / gerekçe
$P(x, Ali)$	$P(Ayse, y)$	$\{x/Ayse, y/Ali\}$
$R(x, x)$	$R(Ali, Ayse)$	Başarısız: x iki farklı sabit olamaz
$Q(f(x))$	$Q(f(A))$	$\{x/A\}$
$Q(f(x))$	$Q(g(A))$	Başarısız: f ve g farklı semboller
$P(x)$	$P(g(x))$	Başarısız: occur check
$P(x, y)$	$P(y, z)$	Örn. MGU $\{x/y, y/z\}$ (bağlama birleşimi uygun uygulanır)

Son satırda x,y,z aynı terim olarak birleşmek zorundadır; farklı ama eşdeğer MGU yazımları mümkündür. Önemli olan mümkün olduğunca az sabit değer dayatmaktır.

A.4 İleri zincirleme: dört kural, üç tur

Bilgi tabanı:

$$\begin{aligned} R_1 &: Student(x) \Rightarrow HasID(x), \\ R_2 &: HasID(x) \wedge PaidFee(x) \Rightarrow CanRegister(x), \\ R_3 &: CanRegister(x) \Rightarrow HasLMSAccess(x), \\ R_4 &: HasLMSAccess(x) \Rightarrow ReceivesMaterials(x), \\ & \qquad Student(Ali), \quad PaidFee(Ali). \end{aligned}$$

Soru: $ReceivesMaterials(Ali)$ doğru mudur?

Tur	Eklenen yeni gerçek
Başlangıç	$Student(Ali), PaidFee(Ali)$
1	$HasID(Ali)$
2	$CanRegister(Ali)$
3	$HasLMSAccess(Ali)$
4	$ReceivesMaterials(Ali)$
5	Yeni gerçek yok: fixed point

Burada R_2'nin çalışması için hem önceki yeni gerçek $HasID(Ali)$ hem de başlangıç gerçeği $PaidFee(Ali)$ gerekir. Artımlı algoritma ikinci turda özellikle R_2'yi kontrol eder. Hedef tur 4'te doğrulanırsa «tek cevap yeter» varsayımıyla daha erken durulabilir.

A.5 Geri zincirleme: aynı problemde hedeften gerçeğe

A.4'ün kurallarını tekrar kullanalım. Hedef $ReceivesMaterials(Ali)$ olduğunda R_4 alt hedef $HasLMSAccess(Ali)$ doğurur. R_3, $CanRegister(Ali)$ alt hedefine; R_2 ise $HasID(Ali)$ **ve** $PaidFee(Ali)$ alt hedeflerine indirger. R_1, $HasID(Ali)$ için $Student(Ali)$ gerçeğine gider. Son iki başlangıç gerçeği bilgi tabanında bulunduğundan bütün hedefler kanıtlanır. Fark: Forward Chaining birçok mümkün sonucu üretebilir, Backward Chaining yalnız bu sorunun bağımlılık zincirini izler.

A.6 CNF dönüşümü — kısa fakat tam örnek

Soru. $\forall x\,[Student(x) \Rightarrow \exists y\, Advises(y, x)]$ ifadesini CNF'ye dönüştürün.

1. Koşulluyu kaldır: $\forall x\,[\neg Student(x) \vee \exists y\, Advises(y, x)]$.
2. Varoluşsal y, evrensel x'e bağlıdır. Yeni Skolem fonksiyonu $AdvisorOf(x)$ seç.
3. Skolemleştirme: $\forall x\,[\neg Student(x) \vee Advises(AdvisorOf(x), x)]$.
4. Evrensel niceleyiciyi gizle: $\boxed{\neg Student(x) \vee Advises(AdvisorOf(x), x)}$.

Yanlış dönüşüm örneği: $\neg Student(x) \vee Advises(A, x)$ ifadesi herkesin *aynı* A tarafından danışmanlık aldığını ima eder; özgün ifade böyle bir zorunluluk içermez.

A.7 Resolution ile çelişki ispatı

Verilenler: Bütün insanlar ölümlüdür ve Sokrates insandır.

$$\forall x\,[Human(x) \Rightarrow Mortal(x)], \qquad Human(Socrates).$$

Hedef $Mortal(Socrates)$ ifadesini kanıtlamaktır.

CNF kümesi:

$$\{\neg Human(x) \vee Mortal(x),\quad Human(Socrates),\quad \neg Mortal(Socrates)\}.$$

İlk ve son tümceyi $x = Socrates$ ile çözümleyince $\neg Human(Socrates)$ kalır. Bu da $Human(Socrates)$ ile çözümlenince $\square$ olur. Dolayısıyla $KB \models Mortal(Socrates)$. Burada hedefin olumsuzlamasının eklenmesi **olmazsa olmaz** adımdır.

A.8 Horn tümcesi ile genel tümce farkı

$P(x) \wedge Q(x) \Rightarrow R(x)$ kesin tümcedir: CNF'de $\neg P(x) \vee \neg Q(x) \vee R(x)$ biçiminde **tek** pozitif literal vardır. Öte yandan $P(x) \vee Q(x)$ ifadesi iki pozitif literal içeren bir tümcedir; kesin tümce değildir. Genel çözümleme her iki türle de çalışabilir, fakat basit ileri/geri zincirleme algoritmalarının tamlık garantisi kesin tümce biçiminde tanımlanmıştır.

A.9 Subsumption ve indeksleme

Elimizde $Knows(Ali, Ayse)$, $Knows(Ali, Mehmet)$, $Brother(Ali, Can)$ bulunsun. $FETCH(Knows(Ali, x))$ sadece `Knows` kovasını tarar; sonuçlar $\{x/Ayse\}$ ve $\{x/Mehmet\}$ olur. Bilgi tabanına $Knows(Ali, x)$ **evrensel bir gerçek** olarak eklenmiş olsaydı özel `Knows(Ali,Ayse)` kaydı mantıksal olarak yeni bilgi vermezdi. Ancak «x herhangi biri» ile «bilinmeyen tek bir kişi»yi karıştırmamak gerekir: varoluşsal kişi için taze sabit kullanılır.

A.10 Eşitlik ve yeniden yazma

Verilenler: $SquareRoot(16) = 4$ ve $Value(SquareRoot(16), Positive)$. Demodulation uygulandığında:

$$\boxed{Value(4, Positive)}$$

çıkar. Ancak semboller üzerinde bu eşitlik bilgisi bulunmasaydı standart yapısal Unification, `SquareRoot(16)` terimini doğrudan `4` ile eşleştiremezdi.

A.11 Tabled logic programming neden yararlı?

$path(a, b)$ alt hedefinin yüzlerce farklı ispat dalında tekrarlandığını düşünelim. Saf geri zincirleme aynı yol aramasını tekrar yapabilir. Tablolu yöntem bu hedefin cevaplarını bir kez saklayıp sonraki çağrılarda yeniden kullanır. Bu, dinamik programlamadaki önbelleğe alma mantığıdır; Datalog'daki döngü kontrolü ve tamlık açısından büyük önem taşır.

A.12 Sınav tipi bütünleşik soru

Bilgi tabanı:

$$\forall x\,[Researcher(x) \Rightarrow WritesPaper(x)],$$
$$\forall x\,[WritesPaper(x) \wedge Accepted(x) \Rightarrow Presents(x)],$$
$$Researcher(Ece), \qquad Accepted(Ece).$$

a) Forward Chaining: $WritesPaper(Ece)$ türetilir, ardından $Presents(Ece)$ elde edilir.

b) Backward Chaining: $Presents(Ece)$ hedefi $WritesPaper(Ece)$ ve $Accepted(Ece)$ alt hedeflerine indirgenir. İkincisi bilinir, birincisi $Researcher(Ece)$'den çıkar.

c) Resolution: Hedef olumsuzlanır: $\neg Presents(Ece)$. İkinci kuralın CNF biçimi:

$$\neg WritesPaper(x) \vee \neg Accepted(x) \vee Presents(x).$$

$x = Ece$ çözümlemesiyle $\neg WritesPaper(Ece) \vee \neg Accepted(Ece)$ elde edilir. $Accepted(Ece)$ gerçeğiyle çözümleme yapılınca $\neg WritesPaper(Ece)$ kalır. İlk kuralın CNF biçimi $\neg Researcher(x) \vee WritesPaper(x)$ ve bilinen $Researcher(Ece)$ birlikte $WritesPaper(Ece)$ üretir. Son iki tamamlayıcı literal boş tümce verir. Üç yöntemin mantıksal sonucu aynıdır.

Ek B — Hızlı tekrar, kavram sözlüğü ve kontrol soruları

B.1 Karar rehberi: hangi yöntem ne zaman?

Durum	Uygun yaklaşım	Dikkat edilmesi gereken
Yeni bilgi geldikçe bütün sonuçları hesaplama	Forward Chaining	İlgisiz sonuçlar, tekrar eşleştirme
Tek bir hedefi ispatlama	Backward Chaining	Sonsuz özyineleme, tekrar alt hedef

Durum	Uygun yaklaşım	Dikkat edilmesi gereken
Fonksiyonsuz kesin tümceler, sonlu sabitler	Datalog	Sabit sayısı / veri büyüklüğü
Çok sayıda kısmi kural eşleşmesi	Rete	Eşleşme belleğinin güncellenmesi
Geri aramada tekrar eden alt problemler	Tabling / Memoization	Tablo boyutu ve alan sınırları
Genel FOL tümceleri	Resolution + factoring	CNF, Skolemization, unification
Eşitlik yoğun ispatlar	Paramodulation / eşitlik birleştirmesi	Yeniden yazma yönü / teori seçimi
Sayısal aralık ve kısıtlar	CLP	Kullanılan kısıt çözücünün kapsamı

B.2 Kavram sözlüğü

İngilizce terim	Türkçe karşılık ve öz
Inference	Çıkarım: bilgiden sonuç üretme
Soundness	Sağlamlık: türetilen sonuçlar mantıksal olarak geçerli
Completeness	Tamlık: bütün gerektirilen hedefler ispatlanabilir
Universal Instantiation	Evrensel özelleştirme: genel kuralı ground terime uygula
Existential Instantiation	Varoluşsal özelleştirme: taze tanık sabiti oluştur
Propositionalization	Önermeselleştirme: ground FOL örneklerini önerme gibi ele alma
Semidecidable	Yarı karar verilebilir: olumlu durumlarda ispat bulunur
Unification	Birleştirme: iki ifadeyi eşleştiren yerine koymayı bulma
Most General Unifier	MGU: en az kısıtlayıcı birleştirme
Occur Check	Değişkenin atanacak terim içinde bulunma kontrolü
Standardizing Apart	Bağımsız değişkenleri ayrı isimlerle adlandırma
Generalized Modus Ponens	Genelleştirilmiş Modus Ponens: değişkenli çıkarım
Lifting	Ground çıkarımı değişkenli düzeye yükseltme
Definite Clause	Tek pozitif literal içeren tümce
Datalog	Fonksiyon sembolsüz kesin tümce dili
Fixed Point	Sabit nokta: yeni gerçek çıkmayan KB
Pattern Matching	Örüntü eşleştirme: kural öncüllerini gerçeklere uydurma
Rete	Kısmi eşleşmeleri saklayan ağ tabanlı mekanizma

İngilizce terim	Türkçe karşılık ve öz
Magic Sets	Hedefle ilgili bağlamaları öne çıkaran dönüşüm
AND/OR Search	Bütün koşullar / alternatif kurallar araması
Prolog	Geri zincirleme odaklı mantıksal programlama dili
Choice Point	Alternatifler için geri dönülecek seçim noktası
Trail	Değişken bağlamalarını geri almaya yarayan yığın
Tabled Logic Programming	Alt hedef cevaplarını saklayarak geri aramayı iyileştirme
Closed-World Assumption	Çıkarılamayan bilgi yanlış kabul edilir
CNF	Birleşimsel normal biçim: tümcelerin birleşimi
Skolemization	Varoluşsal niceleyicileri sabit/fonksiyonla kaldırma
Resolution	Tamamlayıcı literalleri eleyerek çıkarım yapma
Factoring	Birleştirilebilir yinelenen literalleri teke indirme
Refutation	Çürütme: hedefin olumsuzundan çelişki çıkarma
Herbrand Universe	Terim sembollerinden oluşan bütün ground terimler
Lifting Lemma	Ground çözümlemeyi FOL düzeyine aktarma
Demodulation	Yönlü eşitlik ile terim yeniden yazma
Paramodulation	Genel tümcelerde eşitlikle sonuç üretme
Unit Resolution	Her adımda birim tümce kullanılan Resolution
Subsumption	Daha genel tümcenin gereksiz özel tümceyi kapsaması

B.3 Kendi kendini değerlendirme soruları

1. Evrensel ve varoluşsal özelleştirme arasında sabit seçimi bakımından nasıl fark vardır?
2. Bilgi tabanında fonksiyon sembolü bulunması önermeselleştirme için neden sorun yaratır?
3. Generalized Modus Ponens'in normal Modus Ponens'e göre ek yeteneği nedir?
4. $UNIFY(P(x), P(F(x)))$ neden standart sonlu-terim mantığında başarısızdır?
5. MGU'nun en genel olması neden önemlidir?
6. Datalog'da ileri zincirleme neden sonlu ground sembol dağarcığıyla sonlanır?
7. Rete algoritması hangi gereksiz hesaplamayı azaltır?
8. Forward ve Backward Chaining'in aynı suç örneğindeki ispat ağacı yönleri nasıl farklıdır?
9. Prolog'da kural sırası neden sonsuz döngüye yol açabilir?
10. Skolem sabiti yerine Skolem fonksiyonunun gerektiği bir cümle yazınız.
11. Resolution ispatında neden hedefin olumsuzlaması bilgi tabanına eklenir?
12. Binary resolution ve factoring birlikte neden önemlidir?
13. Herbrand teoremi sonsuz ground terimler arasında hangi sonlu özelliği garanti eder?

14. Lifting Lemma'nın ana fikrini bir cümleyle açıklayınız.
15. $A \lor B$ ve $\neg B \lor C$ için resolvent nedir?

Kısa cevap anahtarı: (1) UI bilinen terimi, EI yeni sabiti seçer. (2) Sonsuz iç içe terim üretilebilir. (3) Önce unification yapar. (4) x kendi atanacak teriminde geçer. (5) Gereksiz sabit kısıtı eklemez. (6) Olası ground atomlar sonludur. (7) Kısmi eşleşmeyi yeniden kurmayı. (8) Forward aşağıdan, backward yukarıdan başlar. (9) Özyinelemeli alt hedef çözülmeden ikinci kurala ulaşılamaz. (10) $\forall x \exists y\, R(x, y)$. (11) Çelişki yoluyla gerektirmeyi ispatlamak için. (12) Tekrarlı birleştirilebilir literallerin elenmesiyle tamlık elde edilir. (13) Tatmin edilemeyen sonlu ground alt küme. (14) Ground ispatların FOL'da karşılıkları bulunur. (15) $A \lor C$.

B.4 Bölümün ana sonucu

Birinci dereceden mantıkta kanıta giden ortak yapı **niceleyiciler + yerine koyma + unification + çıkarım kuralları** üzerine kuruludur. Önermeselleştirme fikir olarak basit fakat pahalıdır. Lifting aynı gereksiz örnekleri üretmeden çıkarım yapmamıza imkân verir. **Forward Chaining** bilinen gerçekleri genişletir, **Backward Chaining** hedefi alt hedeflere indirger, **Resolution** ise daha genel FOL ifadelerini CNF üzerinden çelişki yoluyla ispatlar. Sağlamlık mantıksal geçerliliği; tamlık, çıkarsanabilir bir sonucun bir ispatının bulunabilmesini güvence altına alır. Ancak genel FOL'un yarı karar verilebilirliği nedeniyle her olumsuz sorgu için sonlu sürede cevap garantisi yoktur.

Kaynak notu: Temel başlık sırası, numaralı kurallar, kitap ispatları ve Şekil 9.4, 9.5, 9.7, 9.9, 9.10, 9.11, 9.12 ve 9.13 kullanıcının yüklediği *Artificial Intelligence: A Modern Approach*, 3. baskı, 9. bölüm metin ve görsellerinden hazırlanmıştır. «Ek A» soruları, ek karşılaştırma grafikleri ve basitleştirilmiş görsel şemalar eğitim amaçlı tamamlayıcı içeriktir. PDF, salt birebir sayfa dizgisi değil, çeviriyi öğretici açıklama ve yeniden düzenlemeyle birleştiren Türkçe çalışma dokümanıdır.